

Mid-Term Report

高振明/Zhenming Gao

January 8, 2026

1 Current Status

As previously mentioned (2025/10/15) in my opening report, project-related coding is almost completed though there are certain aspects of *Tigris*¹ that could use a little bit more optional polishing. Specifically, we have completed

- the coding of *Tigris*' both as a language and a prover.
- passed the major modular feature test at `examples/`.

What's only left is

- the thesis.

Below, we present (non-definitive) **language specifications** and brief **internal design notes** of the language as **proof-of-progress**, which, shall include the following (based on that of OCaml):

- PL side (emphasized, in Lean):
 1. *Tigris*: The Programming language
 - (a) Lexical conventions,
 - (b) Values,
 - (c) Names,
 - (d) Type expressions,
 - (e) Patterns,
 - (f) Expressions, Declarations and Bindings
 - (g) Type(class) definitions,
 - (h) Implementation details i.e. Lexing, Parsing.
 2. Type System
 - (a) Core calculus
 - (b) Typing discipline
 - (c) Exhaustiveness checking
 - (d) Typeclass instance resolution
 - (e) System F IR & elaboration
 3. The *Lambda* IR
 - (a) Lowering
 - (b) Optimizations

¹Repository at <https://github.com/notch1p/tigris>

- (c) Incremental APIs
- 4. Codegen
 - (a) Backend: Common Lisp (SBCL)
 - i. Memory Representation
 - ii. Runtime
- 5. FFI Utilities (SBCL)
- 6. CLI Interfaces (Interpreter, Compilers)
 - (a) Tigrisi: IR-based REPL
 - (b) Tigrisl: Compiler
 - (c) Tigris: (Legacy, deprecated) Source-based REPL
- 7. Misc: Dependently-typed table printer
- Prover side (in Haskell):
 - 1. *Tigrisd*: The language itself
 - (a) Lexical conventions,
 - (b) Values,
 - (c) Names,
 - (d) Type expressions,
 - (e) Constants,
 - (f) Patterns,
 - (g) Expressions,
 - (h) ...
 - 2. Type System
 - (a) ...

Nomenclature In short: $TinyTheorem = Tigris + Tigrisd$

This document, while being a progress report only, is also an incomplete draft of the thesis' core.

Contents

1	Current Status	1
2	The <i>Tigris</i> language	6
2.1	Lexical conventions	6
	Encoding	6
	Blanks	6
	Comments	6
	Identifier	6
	Integer literals	6
	String literals	6
	Reserved symbols and keywords	6
2.2	Value	7
2.3	Names	7
2.4	Type expressions	8
	Type variables	8
	metavariables	9
	skolem-TVs	9
	Function types	9
	Product types	9
	Constructed types	9
	arbitrary recursive types	9
2.5	Patterns	9
	variable & wildcard patterns	10
	literal patterns	10
	parenthesized patterns	10
	variant patterns	11
	Record patterns	11
2.6	Expressions	11
	Basic expressions	12
	Shorthand syntax	13
	Curried applications & definition	13
	Function application	13
	Lambda abstraction	13
	Function definition	13
	Let-bindings & expressions	14
	Nomenclature	14
	Local non-recursive definitions	14
	Local recursive definitions	15
	Global definitions	15
	Type inference & generalization	16
	Control structures	16
	Sequential operations	16
	Conditional expressions	16
	Case analysis	16
	Operators	17
	extern definition	19
2.7	Type, typeclass and instance definitions	19
	datatype definition	19

Typeclass definition	20
Instance definition	21
2.8 Wrapping Up: Brief implementation notes	22
Lexing & Parsing	22
Typechecker	23
3 Type System of the <i>Tigris</i> language	24
Nomenclature	24
Predicates	24
<i>Strict</i> Higher-Kinded Types	24
Limited Rank-N polymorphism	25
Constraint generation & Solving	25
Let-Polymorphism	25
3.1 Formal Grammar	25
3.2 Judgments & Inference rules	27
Unification	27
Constraint Solving	28
Instantiation	28
Generalization	28
Expression Typing	28
Patterns	31
3.3 Instance resolution	33
Instance metadata	33
Head Unification	33
Resolution Algorithm	34
Termination	36
Semantic coherence	36
Evidence generation	36
Context propagation	36
3.4 System F elaboration	37
Dictionary memoization	38
4 The <i>Lambda</i> IR, CPS, Codegen & FFI	40
4.1 The <i>Lambda</i> IR	41
Definition of <i>Lambda</i> IR	41
Value	41
Rhs	41
Stmt	41
Tail	41
LExpr	42
Shortpath: Constructors & Primitive operations	44
Dictionary projections	45
Function, function applicaton & uncurrying	45
Pattern matching	47
4.2 Closure conversion of the <i>Lambda</i> IR	48
Closure conversion	48
The middle ground	48
Function calling convention	48
4.3 Optimizations of the <i>Lambda</i> IR	50
Optimizations	50

	Constant folding & propagation	50
	Copy propagation & dead code elimination	51
	Tailcall	51
	Incremental APIs and the <i>Tigrisi</i> IR interpreter	53
	the <i>Tigrisi</i> IR interpreter	53
4.4	CPS transformation	56
	CPS transformation	56
4.5	Backend: Common Lisp (SBCL) & FFI	61
	The SBCL backend	61
	The runtime system	63
	FFI	64
	tigrisl	67
5	PP: The <i>dependently-typed</i> table printer	68
6	<i>Tigrisd</i>: a lightweight theorem prover	72
6.1	The basics	72
	Indexed family	72
	function	72
7	Summary	73

Symbols	Keywords									
→ ;; ⇒	mutual	infixl	infixr	match	then	let	fun			
, _ : ∀	extern	class	forall	data	prefix	postfix	in			
	type	with	instance	else	and	rec	if			

Table 1: Reserved symbols and keywords

2 The *Tigris* language

Tigris is a purely functional, flexible, static typed language with a focus on readability and syntactic uniformity that takes inspiration from Lean, OCaml and Haskell.

2.1 Lexical conventions

Below we use an augmented form of BNF (not to be confused with ABNF) that accepts additional POSIX regex (specifically, `+[^ ]`) on the RHS.

Encoding Tigris sources are expected to be UTF-8 encoded text.

Blanks Tabs, spaces, CR/LFs are considered blanks. Blanks are ignored and thus do not affect the semantic of the program.² Blanks separate adjacent identifiers, literals and keywords in order to prevent being parsed as one single identifier.

Comments One of `--` (Lean/Haskell/Lua style), `NB.` (J style) starts a *single-line* comment. Comments are treated as if they are blanks. Note that currently multiline comments are not supported.

Identifier Almost the same as OCaml/Haskell/Lean.

```

<ident>      ::= (<letter> | _) <rest>*
<upper-ident> ::= <uppercase> <rest>*
<lower-ident> ::= <lowercase> <rest>*
<letter>     ::= <upper-letter> | <lower-letter>
<rest>       ::= <letter> | 0..9 | _ | ' | ! | ?
<lower-letter> ::= a..z
<upper-letter> ::= A..Z

```

To simplify parsing, whether an identifier is capitalized or not carries additional information. In contrast, no distinction is made between `!?`. This is permitted to facilitate the writing of idiomatic codes such as ending a nullable (**Option** `a`) or a non-local exiting function identifier in `?` or `!`, respectively.

Integer literals `<intlit>` ::= `("-" | "+")? 0..9+`, in decimal.

String literals `<strlit>` ::= `"[^"]+"`

Reserved symbols and keywords See [Table 1](#). Lexical ambiguities are resolved according to the *longest-match* rule.

²This is not the case on the prover side.

2.2 Value

Value refers to an expression that cannot be further reduced, a *normal form* so to speak. I.e. $\langle v, \sigma \rangle \Downarrow \langle v \rangle$ where v is one of

Integer numbers The range of an integer is implementation-dependent, that is, as of now, both SBCL backend and the interpreter (in Lean) support arbitrary-precision arithmetic.

Strings String values are finite sequences of characters, the size of which is implementation-dependent, that is, as of now, SBCL backend supports strings containing up to 2^{45} chars.

Tuples n -tuple is written $(v_1, \dots, v_n)$.³

Records Records are syntactic sugars of inductive types, written $\{field_1 = v_1, \dots, field_n = v_n\}$.⁴ Currently, no type-directed parsing is implemented because of potential major refactors. Thus for records with same field labels, regardless of types, an ambiguous parsing warning is thrown but can be avoided with type ascription (same with patterns):⁵

```
type Point = {x : Int, y : Int}
type Point' = {x : Int, y : Int}

-- [WARN] ambiguous record literal (using default 'Point'), consider adding type ascription;
-- as we currently do not have type-directed parsing.
-- candidates are [(Point, #[x, y]), (Point', #[x, y])]
let _ = {x = 1, y = 2}

let _ = ({x = 1, y = 2} : Point') -- success
```

For overlapping labels, a *longest-match* rule is used:

```
type Point = {x : Int, y : Int}
type Point3D = {x : Int, y : Int, z : Int}
let (x, y) = ( {x = 1, y = 2} -- x : Point
, {x = 1, y = 2, z = 3} ) -- y : Point3D
```

Constructed values refer to values that are constructed by a *value constructor* (or simply unqualified, *constructor*) of some inductive type. It is also called a *variant*. Constructors are like functions except they may not have parameters: for some constructor C_i , a variant is either C_i or $C_i \text{ fid}_1 \dots \text{fid}_n$. The following variants are built-in:

Constructors	Types
true	Bool
false	Bool
()	Unit

Functions refers to the mappings from values to values. Details explained later.

2.3 Names

Identifiers are used to reference *language objects* by name: [Table 2](#)

³Tuples are specially treated instead of the usual `Prod` encoding.

⁴Typeclass instances are syntactic sugars of records.

⁵This annotation must be “next to” the record literal as it is processed by the parser rather than the typechecker.

Category	First letter
Values	any
Constructors	uppercase
Type constructors	uppercase
Record fields	any
Bound type variables	lowercase

Table 2: Case difference of identifier categories

Operator	Associativity
Type application	left
Product type (*)	right
Arrow type ($\rightarrow$)	right

Table 3: type operator associativity

2.4 Type expressions

<code><type-expr></code>	<code>::= <type-app> (-> <type-expr>)?</code>	(arrow type)
<code><type-atom></code>	<code>::= <ident></code>	
	<code> "(" <type-expr> ")"</code>	(parenthesized)
	<code> <type-ctor></code>	
	<code> sepBy(* ×, <type-expr>)</code>	(product type)
	<code> <type-args></code>	
<code><type-app></code>	<code>::= <type-app>? <type-atom></code>	(type application)
<code><type-ctor></code>	<code>::= <upper-ident></code>	
<code><bound-ty></code>	<code>::= <lower-ident></code>	
<code><type-binder></code>	<code>::= <bound-ty></code>	(type binder)
	<code> "(" <bound-ty> : <intlit> ")"</code>	
<code><type-args></code>	<code>::= <bound-ty> <type-expr></code>	(type argument)
<code><qualified></code>	<code>::= "[" sepBy1(",", (<bound-ty> <type-ctor>) <type-expr>+) "]"</code>	
<code><scheme></code>	<code>::= { FORALL <type-binder>+ }? <qualified>? ", " <type-expr></code>	
FORALL	<code>::= forall ∀</code>	

Note that, for juxtapositional type application⁶, product type and arrow type, we have the following precedence & associativity table (higher comes first): [Table 3](#)

Type variables type variables (TVs) are identifiers with a lowercase beginning. The type (kind) of a TV is determined explicitly to aide the absent of kind inference. A TV denoted $(\alpha : n)$ is $\text{Type} \rightarrow \text{Type}$ which otherwise is Type .⁷ In datatype definitions, type variables are always bounded, appearing at parameter position as well a subsequent type expression on the RHS. Note that validity checking of type expressions are mostly done at parsing time i.e. If some TV is unbound, a parsing error is immediately thrown. In type constraints a TV may be one of:

⁶similar to the a/b-type implementation in Haskell 2010 report.

⁷Note that however we do not allow curried HKT i.e. $(\alpha : 2)$ is instead, $\text{Type} \times \text{Type} \rightarrow \text{Type}$. Because of this, we shall refer to it as *arity* rather than *kind* from now on

metavariables Also called *MVs*, denoted $?m.N$ where N is a counter. Refers to a unspecified type that *must* be either instantiated⁸ by a type or generalized when leaving the scope⁹ MVs must not appear in the final scheme.

skolem-TVs Also called *skolems*, denoted $?sk.A$ where A is the original TV. Skolem variables are TVs produced by the process of skolemization. These TVs are essentially type constructors¹⁰ that may *not* be instantiated. This allows the typechecker to catch more errors:

```
instance : Functor Option =
  { map -- forall a b, (a -> b) -> Option a -> Option b
    | f, None => None
    -- [ERROR] Can't unify type ?sk.a with ?sk.b.
    | f, Some a => Some a
  }
```

Results in a typing error because both a and b are skolemized when entering the body of `map` and they may not be instantiate to each other. Here, we assume that the user forgot to apply `f`, which, should instead be `Some (f a)`. This the rigid nature of skolem-TVs allows the typechecker to catch this human error for us.

Note that, there is currently no implicit inference of type variables. They can only be introduced at a $\forall$ binder or datatype declarations.

Function types The type expression $ty_1 \rightarrow \dots \rightarrow ty_n$ denotes the type of functions mapping of $ty_1, \dots, ty_{n-1}$ to results of type ty_n .

Product types The type expression $ty_1 \times \dots \times ty_n$ denotes the type of tuples whose elements are of $ty_1 \times \dots \times ty_n$ respectively.

Constructed types Similar to TVs, type constructor has arity as well. *<type-ctor* is a type constructor with no parameter and a type expression. Otherwise, a type constructor must be fully applied.

arbitrary recursive types is rejected thus it is not possible to type `fun x -> x x`.

2.5 Patterns

Patterns, as in pattern matching, are templates of instructions used to *destructure* data structures of some shape. Patterns can be enriched by custom operators (described later) if they are associated with constructors however they are reduced to one of, or the composition of the primitive patterns below at parsing-time:

<pre><pat> ::= <pat-left> <infix-op> <pat> <pat-left> <prefix-op> <pat> <pat> <postfix-op></pre>	<pre>infix operator enriched prefix operator enriched postfix operator enriched</pre>
<pre><pat-left> ::= <pat-atom></pre>	

⁸The word *instantiate* is used throughout this text while its usage is not all well-defined: Sometimes, instantiate means *specialize*, where TVs are instantiate to concrete types; sometimes, it only indicates the substitution of a TV.

⁹that is, the whole toplevel phrase.

¹⁰that is, encoded as TCons to prevent illicit unification

Operator	Associativity
Constructor application	left
Prefix operator	
Postfix operator	
Infix operator	left
Infix operator	right

Table 4: Precedence of operators in patterns

	<code><ctor></code> many1(<code><pat-atom></code>)	variant
<code><pat-atom></code>	<code>::= <lower-ident></code>	variable
	<code><ctor></code>	constructor
	<code>"_"</code>	wildcard
	<code>"{" sepBy1(",", <pat-field>) "}"</code>	record pattern
	<code>"(" sepBy1(",", <pat>) ")"</code>	product pattern
	<code>"(" <pat> : <type-expr> ")"</code>	ascribed pattern
	<code><literal></code>	
<code><pat-field></code>	<code>::= <field> = <pat></code>	

Note that the arity of a constructor must match the number of sub-patterns associated with it i.e. constructor in this context must not be partially applied. Mixfix (i.e. {pre,in,post}-fix) operators have the following precedence in patterns (higher comes first): [Table 4](#)

variable & wildcard patterns binds the name to the value, e.g.:

```
let fst (x, _) = x
```

Unlike other languages, pattern are not linear. A variable can be bound several times by a given pattern and overwrites previous bindings. Beware of the semantic confusion:

```
let prodEq : ∀a b, a × b → Bool
  | (x, x) ⇒ true
  | (x, _) ⇒ false
-- [WARN] Found redundant alternatives at
-- 2) #[(x, _)]
```

It is not possible to decide the equality relation between two parts of a data structure through arbitrary patterns. Use the method (`=`) of typeclass `Eq` for that matter.

literal patterns test the equality of the constants.

```
let strToBool : String → Bool = fun
  | "true" ⇒ true
  | "false" ⇒ false
-- [WARN] Partial pattern matching, an unmatched candidate is
-- [_]
```

parenthesized patterns come into effect when mixing with operator-enriched patterns:

```
type List a = Nil | Cons a (List a)
infixr 90 "::" ⇒ Cons
```

```
let f1 (x :: y :: _) = (x, y)
let f2 ((x :: y) :: _) = (x, y)
```

f1 and f2 are vastly different, according to *Tigris*' System F IR:

```
let (::) : ∀ α, α → List α → List α = (λ α. Cons@α)

let f1 : ∀ α, List α → α × α =
  (λ α. fun ?x₀ : List α ⇒ match ?x₀ with | Cons x (Cons y _) ⇒ (x , y))

let f2 : ∀ α, List (List α) → α × List α =
  (λ α. fun ?x₀ : List (List α) ⇒ match ?x₀ with | Cons (Cons x y) _ ⇒ (x , y))
```

variant patterns are of the form $C_i p_1, \dots, p_n$. It matches all variants whose constructor equals to C_i and whose subpatterns matches $p_1, \dots, p_n$. An arity mismatch error is signaled if the number of subpatterns is unexpected by the constructor.

Record patterns are similar to record literals that will encounter ambiguous parsing if there exists records with same field labels.

```
type Point = {x : Int, y : Int}
type Point' = {x : Int, y : Int}

let sum {x, y} = x + y
-- [WARN] ambiguous record pattern (using default 'Point'), consider adding type ascription;
-- as we currently do not have type-directed parsing.
-- candidates are [(Point, #[x, y]), (Point', #[x, y])]
let sum' ({x, y} : Point') = x + y
```

Since records are syntactic sugar of inductive types, same applies to record patterns. The denotational semantic is

$$\llbracket \text{type } T \text{ args}^* = \{(fid : \sigma)^*\} \rrbracket := \llbracket \text{type } T \text{ args}^* = T \sigma^* \rrbracket$$

where tm^* denotes the *splice* of tm , a sequential structure. Thus, a field can be extracted using the ordinary constructor application pattern:

```
class Functor (m : 1) =
  {map : forall a b, (a → b) → m a → m b}
let getMap (Functor f) = f
```

yields IR

```
let getMap : ∀ α, Functor α → ∀ a b, (a → b) → α a → α b =
  (λ α. fun ?x₀ : Functor α ⇒ match ?x₀ with | Functor f ⇒ f)
```

2.6 Expressions

Expressions are the core of *Tigris*. Colloquially, we define the following metavariables:

- p_i denotes the i -th pattern in a pattern matching branch.
- C_i denotes the i -th constructor in an inductive type definition
- e, e', e_i denotes an arbitrary expression whose evaluation value depends on the context.

- v, v', v_i denotes a irreducible value.
- $a, b, c, d, e, x, y, z, w$ denotes a binder, or an identifier.
- f, g, h denotes an identifier and semantic-wise, a function.
- $\llbracket \rrbracket$ denotes blackboard style brackets, sometimes used as delimiters (if usable ordinary delimiters have run out) in metalanguage.
- greek letters mostly denotes types. Greek letters with a bar on them $\bar{\alpha}$ denotes a list of those letters, with implicit numbering $1, \dots, i$.

A *Tigris* expression will, eventually, after exhaustive application of transformations, be translated into the core calculus **Expr** defined in `types.lean`.

<code><expr></code>	<code>::= <expr-infix></code> <code> (<expr-infix> : <type-expr>)</code>	ascription
<code><expr-infix></code>	<code>::= <expr-left> <infix-op> <expr-infix></code> <code> <prefix-op> <expr-left></code> <code> <postfix-op> <expr-left></code> <code> <expr-left></code>	infix operator prefix operator postfix operator
<code><expr-left></code>	<code>::= fun (<expr-binder>+ ARROW <expr> <expr-match>)</code> <code> let "rec"? <pat'> = expr in <expr></code> <code> let "rec"? <let-binder> sepBy("and", <let-binder>)</code> <code>in <expr></code> <code> if <expr> then <expr> else <expr></code> <code> rec <expr></code> <code> match sepBy1(",", <expr>) with <expr-match></code> <code> <expr-funapp></code>	lambda abstraction pattern matching let let(rec) expression conditional fixpoint; deprecated pattern matching
<code><expr-funapp></code>	<code>::= <expr-funapp>? <expr-atom></code>	function application
<code><expr-atom></code>	<code>::= <literal></code> <code> <expr-ctor></code> <code> <lower-ident></code> <code> "(" sepByN(",", <expr>) ")"</code> <code> "{" sepBy1(",", <expr-field>) "}"</code>	variables unit $N = 1$ parenthesized <code><expr></code> $N \geq 2$ product record literal
<code><expr-ctor></code>	<code>::= <upper-ident></code>	constant constructors
<code><expr-match></code>	<code>::= (" " sepBy1(",", <pat>) ARROW <expr>)+</code>	pattern match body
<code><expr-field></code>	<code>::= <field> = <expr></code>	
<code><expr-binder></code>	<code>::= <id> <pat'></code>	binders
<code><pat'></code>	<code>::= "(" <pat> ")"</code> <code> <literal></code>	parenthesized <code><pat></code> except literal patterns
<code><let-binder></code>	<code>::= <pat'> = <expr></code> <code> <ident> <expr-binder>* <type-expr>? <let-rhs></code>	single-pattern binding value/function binding
<code><let-rhs></code>	<code>::= "=" <expr> <expr-match></code>	let binding RHS
<code><literal></code>	<code>::= <intlit> <strlit> "true" "false"</code>	
<code>ARROW</code>	<code>::= "⇒" "→"</code>	

Note that, the `ARROW` lexer parses both lexeme $\Rightarrow$ and $\rightarrow$, only the first should be used, the alternative is to add some form of ML compatibility.

Basic expressions are all expressions list in `<expr-atom>` with the addition of function shorthand syntax.

Shorthand syntax is a Lean-like syntactic sugar for parenthesized expressions, within which, an underscore (`_`) or `cdot` (`·`) is transformed in preorder¹¹ against the expression to a function binder. This is similar to Haskell’s operator sectioning. Respects precedence, e.g.

	<code>fun x ⇒ x + 1</code>
<code>(_ + 1)</code>	<code>fun x y ⇒</code>
	<code>let tmp = 2 / y in</code>
<code>(_ + 1 * 2 / _)</code>	<code>let tmp = 1 * tmp in</code>
	<code>x * tmp</code>
<code>(id _)</code>	<code>fun x ⇒ (id : forall a, a -> a) x</code>
(a) shorthand syntax	(b) desugared results

Parenthesized expressions can also contain type constraint, as in the example above.

Curried applications & definition Functional values are curried.¹² i.e. partial application is allowed and is in practice frequently used in idiomatic functional programming.

Function application associates to the left. It is denoted by juxtaposition of expressions. *Tigris* is call-by-value (CBV), thus the expression $e\ e'$ evaluates e to a functional value first, then e' .

Lambda abstraction takes the form of `fun  $p_1 \cdots p_n \Rightarrow e$`

Function definition has two syntactic forms:

$$\begin{array}{l}
 \text{fun} \\
 | \ p_{1,1}, \dots, p_{1,m} \Rightarrow e_1 \\
 \vdots \\
 | \ p_{n,1}, \dots, p_{n,m} \Rightarrow e_n
 \end{array} \tag{1}$$

$$\text{fun } p_1 \cdots p_m \Rightarrow e \tag{2}$$

Both forms are syntactic sugar that are translated and, equivalent to

$$\begin{array}{l}
 \text{fun } x_1 \Rightarrow \cdots \text{fun } x_n \Rightarrow \\
 \text{match } x_1, \dots, x_n \text{ with} \\
 | \ p_{1,1}, \dots, p_{1,m} \Rightarrow e_1 \\
 \quad \text{applies to form (1)} \\
 | \ \overbrace{p_{2,1}, \dots, p_{2,m}} \Rightarrow e_2 \\
 \vdots \\
 | \ p_{n,1}, \dots, p_{n,m} \Rightarrow e_n
 \end{array}$$

Syntactic intuition shows that canonically, a lambda abstraction only takes one (1) parameter.

¹¹i.e. DFS.

¹²Constructors and functions form an *exhaustive partition* of functional value.

Let-bindings & expressions The `let` and `let rec` language constructs bind value names either locally or globally, together with `in` `<expr-body>`, they form expressions.

Nomenclature Here, a *let-binding* describes the $p_n = e_n$ portion, and-branches are implicitly assumed until mentioned otherwise; A *let-expression* includes the *body* portion as well.

Local non-recursive definitions the construct `let  $p_1 = e_1$  and  $\dots$  and  $p_n = e_n$  in  $e$` evaluates¹³ $e_1 \dots e_n$ and matches the results against pattern $p_1 \dots p_n$. If matching succeeds, e is evaluated in the lexical scope enriched by the bindings produced during matching, whose value is returned as the value of the whole let expression.¹⁴ Pattern matching let-expression is translated to a `match .. with` construct.

The canonical let-binding described above can be used to bind functional values though syntactic sugar exists: In a let expression, function may be defined as `let  $f$   $p_1 \dots p_n = e$` , e.g.

```
let prodSum (x, y) z w = x + y + z + w
and x = 1
and y = 2
in prodSum (x, y) 3 4

let (x, y) = (1, 2) in x + y

let 0 = 1

let compose f g x = f (g x) in
let add20 = (_ + 20) in
let minus30 = fun x => x - 30 in
compose add20 minus30 50
```

In the third example, a warning is thrown;

```
[WARN] Partial pattern matching, possible cases such as [true] are ignored
```

At runtime, the nonrecoverable exception MATCH-FAILURE is thrown:

```
debugger invoked on a MATCH-FAILURE in thread
#<THREAD tid=145807 "main thread" RUNNING {1200030003}>:
  No branch can be matched against #[Toplevel]
```

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):

```
0: [ABORT] Exit debugger, returning to top level.
```

```
(|__start|)
  source: (FUNCALL #'|main| NIL #'IDENTITY)
0]
```

¹³virtually (in the context of a typechecker), in parallel; In practice, this is implementation-dependent. Cf. nested let.

¹⁴no sequential binding is performed. A name bounded to e_i is unbounded in e_{i+1} .

Local recursive definitions are introduced by the `let rec` construct:

$$\text{let rec } f_1 \ p_1 \cdots p_n = e_1 \ \text{and} \cdots \text{and } g \cdots = e_n \ \text{in } e$$

To grossly simplify the feature of `let rec`: the names are considered already bound when the expressions e_1 to e_n are evaluated. In practice, the importance of this construct is obvious as this is the only way to facilitate (mutual) recursion in *Tigris*.

```
mutual
type Tree a = Empty | Node a (Forest a)
type Forest a = Nil | Cons (Tree a) (Forest a)
;;

let rec countTree
  | Empty => 0
  | Node _ f => 1 + countForest f
and countForest
  | Nil => 0
  | Cons t f => countTree t + countForest f
```

New users may be tempting to define recursive functions in a more “functional” way, but hesitate as they do not know whether it will work: `let rec f = fun ... and ... and g = fun ... in e`. The answer is yes: Just the same as `let` binding, the canonical form of a `letrec` expression is precisely the above, what we introduced earlier is simply a syntactic sugar. It defines f , g as mutually recursive functions local to e . This nature is reflected in the core calculus as whether recursive or not a `let rec` binding is, it is always transformed into a `Let` node, with the only difference being the RHS of a `let rec` binding is wrapped in `Fix`.

The semantic difference between `let rec` and `let` is minute, yet complicated. We largely follow OCaml’s implementation but is even simpler and more strict.

Remark (Partition heuristic). the RHS of a `letrec` binding must have the form of `fun ...`; Otherwise, it is ruled as a `let` binding.

Using a simple¹⁵ heuristic,¹⁶ we partition a `letrec` group into one that is strictly *recursive*¹⁷ and another one that becomes regular *let-binding*. Additionally, we require the LHS of a `letrec` binding to always be variables.¹⁸ These assumptions are strict enough to rule out some valid programs.

```
let rec x = (1, x) and y = (2, x)
```

```
let rec x = (1, y) and y = 1
```

The first example is invalid in CBV semantics as it creates cyclic products whereas the second one is ruled out though valid. We find it appropriate as though example 2 is invented, its coding style is inherently bad; Besides, the author also struggles to find a convincing example that justifies the preservation of the expressiveness lost here.

Global definitions are just like local ones, but is limited to the toplevel, where a `let(rec)` expression can be free of its body i.e. becomes a standalone `let` binding. Multiple toplevel `let` bindings can optionally be separated with `;;` for clarity.

¹⁵no additional free occurrence checking, or much of the condition checking in the OCaml manual ch. 12.1.

¹⁶one of the condition to be *statically constructive*.

¹⁷in the sense there is no recursive definition of non-functional values.

¹⁸Unlike Haskell, which translate it into a fixpoint application with pattern matching.

```

function k<T, U>(t: T, _: U): T {return t}
const id: <T>(t: T) ⇒ T = (t) ⇒ t
function map<T, U>(f: (t: T) ⇒ U, xs: T[]): U[] {
  let xss = []
  for (let x of xs) {
    xss.push(f(x))
  }
  return xss
}

let sns = map((n) ⇒ n.toString(), [1, 2, 3])
let sns = map toString $ 1 :: 2 :: 3 :: Nil

```

Type inference & generalization occurs in many places. Type inference provides the user with the ability to omit typing ascriptions while maintaining program type correctness (given the program itself is correct). Unlike other languages, *Tigris* has global type inference, enabling typecheckers to type programs with zero (taken literally) ascriptions, this is due to the type system having principal types i.e. the ability to infer a most general type.

Generalization applies to schemes and happens at let binding. It goes together with inference: Unlike other language, where users must write in some special language construct to define a polymorphic function, *Tigris* facilitates automatic generalization of expressions, which means, if an expression is polymorphic, then it *is*, without the need of supplying type ascriptions.

These two features free programmers from the verbose work of typing expressions while maintaining the type correctness of the program.

Control structures

Sequential operations are not builtin, one may substitute it with nested let expressions.

Conditional expressions are of the form **if** e **then** e_1 **else** e_2 ; which evaluates to e_1 if e yields the bool constructor true otherwise e_2 .

Case analysis Also called pattern matching. The expression

$$\begin{array}{l}
 \mathbf{match} \ e_1, \dots, e_m \ \mathbf{with} \\
 | \ p_{1,1}, \dots, p_{1,m} \Rightarrow e_1 \\
 | \ p_{2,1}, \dots, p_{2,m} \Rightarrow e_2 \\
 \vdots \\
 | \ p_{n,1}, \dots, p_{n,m} \Rightarrow e_n
 \end{array}$$

matches the row expression vector against the pattern matrix *in parallel*,¹⁹ yielding the RHS of a row that first succeeds. At compile time, pattern matrices are checked of exhaustiveness and redundancy, using an efficient matrix-modeled algorithm from Maranget:

```

let (1, 2) = (1, 2) -- [WARN] Partial pattern matching, possible cases such as [(_, _)] are ignored

let f
| (1, 2) ⇒ 1 + 2 -- [WARN] Found redundant alternatives at

```

¹⁹Cf. product pattern, which is evaluated in preorder.


```

-- 3) #[(_, y)]
| (x, _) => x
| (_, y) => y

```

If none matches, the nonrecoverable runtime exception `MATCH-FAILURE` is thrown. Pattern matching can be seen as a superset of the `switch` construct found elsewhere. It enables a declarative paradigm to elegantly destructure data structures and is at the core of modern functional programming.

Operators Operators are special strings bound to expressions. They are recognized by the parser and can be chained in expressions.

One feature that set *Tigris* apart from other languages is its ability to define custom operators from a infinite set of strings with associativity and precedence adjustable. A custom operator definition is either a 4-tuple $(op, prec, assoc, impl)$ or a 3-tuple $(op, prec, impl)$, collected from one of the three operator declaration forms below:

```

<infixl-decl> ::= infixl ":"? <op> <prec> ARROW <expr>      ; => (<op>, <prec>, L, <expr>)
<infixr-decl> ::= infixr ":"? <op> <prec> ARROW <expr>      ; => (<op>, <prec>, R, <expr>)
<prefix-decl> ::= prefix ":"? <op> <prec> ARROW <expr>      ; => (<op>, <prec>, <expr>)
<postfix-decl> ::= postfix ":"? <op> <prec> ARROW <expr>     ; => (<op>, <prec>, <expr>)

```

Where `<op>` denotes the operator string and `<op>` itself is a subset of `<strlit>`.

Definition (Operator candidate strings). Unlike Haskell or OCaml, whose `<op>` comes from combinations of a predefined set of operator characters, *Tigris* follows a Lean-like approach and defines `<op>` to be *any string* except:

- those that begin with numbers,
- those that begin with or contain one of `[_ , ( ) [ ] { } [ ] ]`,
- those that, except the beginning, contain `<blank>`,
- those that equal to a reserved operator shown here: [Table 1](#).

Blank characters *surrounding* `<op>` are automatically trimmed.

Precedence of the four operator classes (infixl, infixr, prefix, postfix) with respect to function application is the same as in patterns: [Table 4](#). Although much²⁰ of the operators are left for users to define, some basic operators are defined (though they can be overridden) using the same mechanics.^{21 22} The addition of `$` and `@@` is a clever abuse of precedence and is to facilitate idiomatic function programming techniques. With the lowest precedence, it is possible to avoid parenthesization nested to the right, as in:

```

let _ = f (g (h (x + y)))
let _ = f $ g $ h $ x + y
let _ = f @@ g @@ h @@ x + y

```

²⁰that is, an (almost) infinite amount of potential operators.

²¹Though in practice it is more common to bind operators with methods from typeclass, we chose to keep it simple; Besides, It is also true that we currently do not have a *prelude*.

²²As described in the expression grammar, following the usual practice, language constructs are hardcoded to have lower precedence than operators, thus `let n = 10 in n + x` parses as `let n = 10 in (n + x)` rather than `(let n = 10 in n) + x`.

Operator	Precedence	Class	Implementation
*	70	infixl	Primitive integer multiplication
/	70	infixl	Primitive integer division
+	65	infixl	Primitive integer addition
-	65	infixl	Primitive integer subtraction
=	50	infixl	Primitive integer equality
\$	1	infixr	Function application
@@	1	infixr	Function application

Table 5: Predefined operators

This becomes particularly handy if the argument of some function is a large expression and this technique is also widely used throughout the implementation of *Tigris* and *Tigrisd* themselves. It is also possible to bound a same operator to both prefix and postfix:

```

let not
  | true ⇒ false
  | false ⇒ true
let rec fact
  | 0 ⇒ 1
  | n ⇒ n * fact (n - 1)

```

```

postfix 65 "!" ⇒ fact
prefix 65 "!" ⇒ not
let main = (50!, !(true))

```

But currently it is not possible to do so between prefix and infix or postfix and infix. Common idiomatic use includes defining the Cons constructor of List as (::) and append as (++):

```

type List a = Nil | Cons a (List a)
infixr 67 " :: " ⇒ Cons
let car (x :: _) = x
let cdr (_ :: xs) = xs
let rec map f
  | Nil ⇒ Nil
  | x :: xs ⇒ f x :: map f xs
let foldl1 f xs =
  let rec go xs =
    match xs with
    | x :: Nil ⇒ x
    | x :: y :: xs ⇒
      go (f x y :: xs)
  in go xs

infixl 65 " ++ " ⇒ append
let rec append
  | Nil, ys ⇒ ys
  | x :: xs, ys ⇒ x :: append xs ys

```

Multiple toplevel operator bindings can optionally be separated with ;; for clarity.

extern definition utilizes the FFI facility to declare *external*²³ functions.

```
<extern-decl> ::= extern <ident> <extern-symb> : <scheme>
<extern-symb> ::= <strlit>
```

The extern declaration introduces an *opaque*²⁴ name <ident> with scheme <scheme>, binding itself to an external function named <extern-symb>. Since *Tigris* is a pure language, useful function with side effects can be introduced with FFI. This is considered an advanced feature, interested users shall refer to [subsection 4.5](#) for a detailed explanation.

2.7 Type, typeclass and instance definitions

datatype definition is used to introduce new types to the environment. It binds type constructors to datatypes in either of the forms: inductive types or record types. Though for the latter it is merely a syntactic sugar of the former as mentioned before.

```
<type-decl>      ::= mutual <type-decl1>+ ";" "          (mutual rectype definition)
                  | <type-decl1>
<type-decl1>    ::= TYPE <type-ctor> <type-binder> = <type-constrs>
<type-field>    ::= <ident> : <type-expr>
<type-constrs> ::= <type-record>
                  | ("|" <type-ind>)+
<type-record>  ::= "{" sepBy1(",", <ident> : <type-expr>) "}"
<type-ind>     ::= <expr-ctor> <type-atom>+          (inductive type branch)
TYPE           ::= type | data
```

Note that, the TYPE lexer parses both lexeme type and data, only the first should be used, the alternative is to add some form of Haskell compatibility.

An inductive type has the form

$$\mathbf{type} \ T \ \bar{\alpha} = C_1 \sigma_{1,1} \cdots \sigma_{1,c_1} \mid \cdots \mid C_n \sigma_{n,1} \cdots \sigma_{n,c_n}.$$

This declaration introduces a new type constructor T that takes n parameter *fully*, with one or more value constructors $C_1, \dots, C_n$. The types of the constructors are given by

$$C_i : \sigma_{i,1} \rightarrow \cdots \rightarrow \sigma_{i,c_i} \rightarrow T \ \bar{\alpha}.$$

where the lowercase c_i represents the arity of C_i .²⁵

```
type RGB = {r : Int, g : Int, b : Int}
type Color =
  | Red | Green | Blue | Yellow | Black | White
  | Custom RGB

type Tree a = Lf | Br a (Tree a) a

mutual
type Tree a = Empty | Node a (Forest a)
type Forest a = Nil | Cons (Tree a) (Forest a)
;;
```

²³in the sense that it is not defined within the language.

²⁴in the sense that it has no definition.

²⁵The result type of a constructor must match the inductive type declaration as indexed families are not currently supported.

Type synonyms (equality constraints) and *contexts* (predicates) are currently not supported for type declarations. Note that, *Tigris* currently does enforce strict positiveness in inductive types.

Definition (Strict Positiveness). A position is said to be strictly positive if

1. it is not in a function's argument type, and,
2. it is not an argument of any expression other than type constructors of inductive types.

Since *Tigris* is a programming language, we do not make this restriction as it may rule out some unproblematic type definition.

Typeclass definition Typeclass is the functional counterpart to what is called an *interface* in Object-oriented programming, except, it is more efficient and well-defined, as in methods are dispatched statically, as in enabling ad-hoc/parametric polymorphism in a principled fashion.

```
<class-decl> ::= class <class-ident> <type-binder> = <type-record>
<class-ident> ::= <type-ctor>
```

A class declaration introduces new class methods, whom are globally scoped. Classes are very similar to records because of their implementation²⁶, except it utilizes the typeclass facility. A class declaration takes the form of

$$\text{class } C \bar{\alpha} = \langle \text{type-record} \rangle$$

and the type of the class method f_i is

$$f_i : \forall \bar{\alpha}, \bar{\beta}. \sigma_i$$

where $\bar{\beta}$ is the additional type variables bound at method level²⁷, and σ_i is the type signature of f_i modulo binder. Consider

```
class Eq a =
  {eq : a -> a -> Bool}
class Functor (f : 1) =
  {map : forall a b, (a -> b) -> f a -> f b}
class HEq a b =
  {heq : a -> b -> Bool}

let main = (map, eq, heq)
```

eq has type $\forall a [\text{Eq } a]. a \rightarrow a \rightarrow \text{Bool}$, map has type $\forall f [\text{Functor } f]. \forall a b. (a \rightarrow b) \rightarrow f a \rightarrow f b$, heq has type $\forall a b [\text{HEq } a b]. a \rightarrow b \rightarrow \text{Bool}$; Whereas main, i.e. some program that uses all the methods above without providing additional type ascription to *refine* the class, yields the verbose

$$\forall \alpha \beta \gamma \delta [\text{Functor } \alpha, \text{Eq } \beta, \text{HEq } \gamma \delta] \forall a b. ((a \rightarrow b) \rightarrow \alpha a \rightarrow \alpha b) \times (\beta \rightarrow \beta \rightarrow \text{Bool}) \times (\gamma \rightarrow \delta \rightarrow \text{Bool}).$$

Default methods and superclass are not supported at this moment.

²⁶dictionary passing.

²⁷supported by polymorphic fields.

Instance definition Just as in object-oriented programming which interfaces are always associated with their implementations, so is typeclass. *Instances* are used to implement typeclasses.

```

<inst-decl> ::= instance ":"? <inst-scheme> = "{" <inst-method-decl> "}"
<inst-scheme> ::= {FORALL <type-binder>+}? <inst-context>? ", " <class-ident> <type-expr>+
<inst-context> ::= "[" sepBy1(", ", <qualified>) "]"
<inst-method-decl> ::= sepBy(", ", <let-binder>)

```

For the sake of syntactic uniformity, the RHS of a instance declaration is similar to a record literal, with its field assignments being similar to a let-binding.

```

instance : Eq Int = {eq x y = __eqInt x y} -- __eqInt is primitive, requires η-conversion
instance : Functor Option =
  { map f
    | Some x ⇒ Some $ f x
    | None   ⇒ None
  }
instance HEq String Bool =
  { heq = fun
    | "true", true   ⇒ true
    | "false", false ⇒ true
    | -, -          ⇒ false
  }
instance forall a b [HEq a b], HEq b a = {heq y x = heq x y}
instance forall a [Eq a], HEq a a = {heq = eq}

```

Instance declarations may quantify type variables, Together with contexts, this can be used (as shown in example 4 below) to express the *implication* relations.²⁸ Here, we quantify a, b and an instance $HEq\ a\ b$ and use it as the implementation for $HEq\ b\ a$ to express the assumed *commutative* nature of a heterogenous equality and to express the fact that the homogenous equality relation subsumes the hetero one. Instance resolution is described later. For now, users only need to remember that

1. a type may not be declared as an instance of some class more than once.
2. overlapping instance results in a compile-time ambiguous exception.
3. ambiguous type arising from the use of a method blocks typeclass elaboration and throws a compile-time ambiguous exception.

```

class HAdd a b c = {hadd : a -> b -> c}
instance HAdd Int Int Int = {hadd = add}

-- Ambiguous: HEq ?m6 Int: typeclass elaboration is stuck because of
-- metavariable(3)
--   [?m6]
-- induced by a call to heq. To resolve this, consider adding type ascriptions.
let f' = heq (hadd 2 3) 5

```

In this example, `hadd 2 3` is typed a metavariable, results in HEq 's instance lookup failure.

Although it is entirely possible to infer the context in a instance declaration just the same as in `main` in the above example, we nevertheless require users to provide explicit instance context.

Multiple toplevel type declarations can optionally be separated with `;;`²⁹ for clarity.

²⁸That is, the $\rightarrow$ in first-order logic, or $\vdash$ in metalanguage.

²⁹except mutual block where `;;` is required to close it.

2.8 Wrapping Up: Brief implementation notes

Below we briefly talk about some key points of *Tigris*' implementation, though readers shall expect a more thorough discussion in the thesis.

Lexing & Parsing Lexing stage and Parsing stage are fused. We employ the functional way of *monadic parsing*, which typically uses *parser combinators*, together with the flexible custom operators, an implementation of a fusion of combinatory parsing and Pratt parsing is ultimately used.

In practice, parser combinators are similar to a recursive descent parser or a hand-written one. The grammar is similar to a parsing expression grammar (PEG) which always selects the first production when encountering ambiguous rules. Though it is currently, not a packrat parser which is guaranteed to run in linear time. Backtracking is still extensively used in our case as a result of the our monad class implementation. A parser monad in *Tigris* is defined to be

```
abbrev TParser σ := SimpleParserT Substring Char
          $ StateRefT (PEnv × String) (ST σ)
```

Where σ is a formal argument for the ST-backed parser monad, allowing efficient and proper mutable state to be stored in threadlocals rather than a simulated StateT that uses stack space. The monad transformer stack also has two transformers SimpleParserT and StateRefT to allow the fluent use of State-like methods in ST as well as the efficient use of Substring where we do not allocate new strings and instead uses a pair of start/end pointer as a *view* during parsing.

According to Wikipedia, the two major parser combinators are (given two parsers P and Q)

- the *alternative* combinator $p \oplus q (j) = P j \cup Q j$
- the *sequence* combinator $(p \otimes q) (j) = \bigcup \{Q k \mid k \in P j\}$

This is exactly, captured by the MonadExcept, Alternative class and the Seq class in monadic functional programming, and is implemented as

```
1 instance (σ ε τ m) [Parser.Stream σ τ] [Parser.Error ε σ τ] [Monad m]
2   : Monad (ParserT ε σ τ m) where
3   -- (...)
4   seq f x s := f s >>= fun
5     | .ok s f => x () s >>= fun
6     | .ok s x => return .ok s (f x)
7     | .error s e => return .error s e
8     | .error s e => return .error s e
9
10 instance (σ ε τ m) [Parser.Stream σ τ] [Parser.Error ε σ τ] [Monad m] :
11   MonadExceptOf ε (ParserT ε σ τ m) where
12   throw e s := return .error s e
13   tryCatch p c s := p s >>= fun
14     | .ok s v => return .ok s v
15     | .error s e => (c e).run s
16
17 instance (σ ε τ m) [Parser.Stream σ τ] [Parser.Error ε σ τ] [Monad m] :
18   OrElse (ParserT ε σ τ m α) where
19   orElse p q s :=
20     let savePos := Parser.Stream.getPosition s
21     p s >>= fun
```

```

22 | .ok s v ⇒ return .ok s v
23 | .error s _ ⇒ q () (Parser.Stream.setPosition s savePos)

```

Basic scaffolding and common combinators are already done by Dorais' [lean4-parser](#) and our implementation depends on this package. Additionally, we have defined several combinators are defined to aid the parsing of function application in `utils.lean` (especially, `chainl` and `chainr`):

```

1  /--
2   `chainl1 p op` parses *one* or more occurrences of `p`, separated by `op`. Returns
3   a value obtained by a **left** associative application of all functions returned by `op` to the
4   values returned by `p`.
5   - if there is exactly one occurrence of `p`, `p` itself is returned touched.
6  -/
7  partial def chainl1
8    (p : ParserT ε σ τ m α)
9    (op : ParserT ε σ τ m (α → α → α)) : ParserT ε σ τ m α := do
10    let x ← p; rest x where
11    rest x :=
12      (do let f ← op; let y ← p; rest (f x y)) <&> pure x
13
14  partial def chainr1
15    (p : ParserT ε σ τ m α)
16    (op : ParserT ε σ τ m (α → α → α)) : ParserT ε σ τ m α := do
17    let x ← p; rest x where
18    rest x :=
19      (do let f ← op; chainr1 p op <&> f x) <&> pure x
20  /- like `test`, but non-consuming -/
21  def test' (p : ParserT ε σ τ m α) : ParserT ε σ τ m Bool :=
22    try lookAhead p $> true
23    catch _ ⇒ return false

```

Typechecker There are two typecheckers available in the *Tigris* repository, a legacy, straight implementation of Algorithm W, as well as a constraint solver version that performs better with qualified types and is actually used. We shall only discuss the latter. Typecheckers are run inside the `InferC Monad`:

```

1  abbrev InferC σ := StateRefT CState (EST TypingError σ)
2
3  local instance : MonadLift (Except TypingError) (InferC σ) where
4    monadLift
5    | .error e ⇒ throw e
6    | .ok res ⇒ return res

```

A tone-downed version of GHC's type variable leveling is used to deal with erroneous unification. Pattern matrices are checked at this stage of exhaustiveness and redundancy, using an efficient matrix-modeled algorithm from Maranget.

The typechecker checks type, as well as producing a typed AST, which will then be *elaborated* to a System F-ish IR, defined very similar to the core calculus of *Tigris*, as:

```

1  inductive FExpr where
2    | CI      (i : Int)    (ty : MLType)
3    | CS      (s : String) (ty : MLType)

```

```

4   | CB      (b : Bool)  (ty : MLType)
5   | CUnit   (ty : MLType)
6   | Var     (x : String) (ty : MLType)           -- monomorphic view at site
7   | Fun     (param : String) (paramTy : MLType)
8             (body : FExpr) (ty : MLType)         -- paramTy → body.ty = ty
9   | App     (f : FExpr) (a : FExpr) (ty : MLType) -- result type after application
10  | TyLam   (a : TV) (body : FExpr)              -- type abstraction
11  | TyApp   (f : FExpr) (arg : MLType)            -- type application
12  | Let     (binds : Array (String × Scheme × FExpr)) -- each binding elaborated & wrapped
13            (body : FExpr) (ty : MLType)
14  | Prod'   (l r : FExpr) (ty : MLType)
15  | Cond    (c t e : FExpr) (ty : MLType)
16  | Match   (discr : Array FExpr)
17            (branches : Array (Array Pattern × FExpr))
18            (resTy : MLType)
19            (counterexample : Option (List Pattern))
20            (redundantRows : Array Nat)
21  | Fix     (e : FExpr) (ty : MLType)              -- rec tag
22  | Proj    (src : FExpr) (mname : String)
23            (idx : Nat) (ty : MLType)             -- field projection
24  deriving Repr, Inhabited

```

Using a pretty-printing algorithm similar to Wadler’s, FExpr is pretty printed to have a syntax that is very much similar (except it always denotes signatures and type application) to a subset of *Tigris*. This algorithm is used for subsequent IRs.

From there, we begin the lowering process by lowering it to *Lambda* IR.

Type system, together with typechecking, is a major part of *Tigris*’ implementation, we’ll have a more thorough discussion below.

3 Type System of the *Tigris* language

Nomenclature Throughout this section, the word *constraint* is extensively used. Since we implement a constraint-solver based typechecker, broadly speaking, constraint can reference to

- the equality constraint, which is reflexive, commutative (though mapping is one-way in our case) and transitive.
- the predicate constraint, which is cumulative, as in this type of constraints can accumulate in the scheme’s context.

Though, sometimes, it may also exclusively have the meaning 2). Readers should be able to deduce the meaning from the context as we stick to the broader sense in this text.

Tigris implements a constraint-based Hindley-Milner type system extended with

Predicates are of the form $C\bar{\alpha}$ where C is a class head. These are collected during inference and resolved via dictionary passing after constraint solving.

Strict Higher-Kinded Types refers to the fact that higher-kinded TVs must be fully applied. This is done through the use of KApp which is just enough for abstractions like Functor and Monad, see [footnote 7](#).

Limited Rank-N polymorphism refers to the fact that *schemes* (TSch) can appear inside monotypes $MType$, but the system remains in rank-1 polymorphism through the addition of skolemization during type ascription checking.³⁰ This means when checking $e : \sigma$, we instantiate σ with skolem constants³¹ and verify the inferred type unifies. Intuitively and informally this should be sound as

- skolem-TVs are rigid in the sense that they are unique and cannot be unified with other types.
- skolem-TVs are tracked through a rigid set that is used to reject substitutions which would otherwise escape scope.

Constraint generation & Solving Generation of constraints are handled by inference, which in turn, solves them in one pass. The solver returns a substitution (also called a *solution*) and residual predicates that become part of the generalized scheme.

Let-Polymorphism refers to let-bound variables are generalized over TVs such that $tv \notin \text{ftv } \Gamma$, with predicate constraints becoming part of the scheme's context. Informally, it is correct to state that, program is still typeable even if users may not provide any signature due to the principality of HM systems.

3.1 Formal Grammar

The type system is built on its core calculus, an augmented version of the lambda calculus. To describe its typing discipline, we first present the formal grammar of the core calculus as well as the type expression, among other things.³²

$expr, e$	$::=$	term
	$const$	constant
	$tname$	variable
	$\lambda x. e$	lambda abstraction
	let $x_1 = e_1$ and .. and $x_n = e_n$ in e'	letexp
	$e e'$	function application
	(e_1, e_2)	product
	if e_1 then e_2 else e_3	conditional
	fix e	fixpoint
	match $e_1, \dots, e_n$ with $Pmatrix$	pattern matching
	$e : sch$	type ascription

$pattern, pat$	$::=$	patterns
	$tname$	variable
	$_$	wildcard
	$const$	constant
	(pat_1, pat_2)	product
	$ctor pat_1 \dots pat_n$	constructor application

$Pmatrix$	$::=$	pattern matrix
	$ pb_1 \dots pb_m $	

$patbranch, pb$	$::=$	pattern branch
	$pat_1, \dots, pat_n \rightarrow expr$	

³⁰as **gen** or **generalize** does not generate higher-rank binders.

³¹encoded by TCon i.e. a nullary constructor

³²most grammar, rules, are generated from report.ott using Ott.

$$\begin{array}{lcl}
\textit{tyctor} & ::= & \\
& | & \textit{upperident} \\
& | & \textit{Int} \\
& | & \textit{Bool} \\
& | & \textit{String} \\
& | & \textit{Unit}
\end{array}$$

$$\begin{array}{lcl}
\textit{tyvar}, \textit{tv}, \textit{a}, \textit{b} & ::= & \\
& | & \textit{lowerident}
\end{array}$$

$$\begin{array}{lcl}
\textit{tvset}, \bar{\alpha} & ::= & \text{type variable set} \\
& | & \emptyset \\
& | & \{\textit{tv}\} \\
& | & \bar{\alpha}_1 \cup \bar{\alpha}_2 \\
& | & \bar{\alpha}_1 \cap \bar{\alpha}_2 \\
& | & \bar{\alpha}_1 \setminus \bar{\alpha}_2 \\
& | & \text{ftv}(\Gamma) \\
& | & \text{ftv}(\textit{ty}) \\
& | & \text{ftv}(\bar{\pi}) \\
& | & \text{ftv}(\textit{pred})
\end{array}$$

$$\begin{array}{lcl}
\textit{tyargs}, \textit{tys} & ::= & \text{one or more type arguments} \\
& | & \textit{ty} \\
& | & \textit{ty} \textit{tys}
\end{array}$$

$$\begin{array}{lcl}
\textit{typeexpr}, \textit{ty}, \tau & ::= & \text{type expression} \\
& | & \textit{tyctor} \quad \text{nullary type constructor} \\
& | & \textit{tv} \quad \text{type variable} \\
& | & \textit{ty} \rightarrow \textit{ty}' \quad \text{arrow type} \\
& | & \textit{ty} \times \textit{ty}' \quad \text{product type} \\
& | & \textit{tyctor} \textit{tys} \quad \text{nominal type application} \\
& | & \kappa \textit{tys} \quad \text{higher-kinded TV head application}
\end{array}$$

$$\begin{array}{lcl}
\textit{predicate}, \textit{pred}, \pi & ::= & \text{predicate} \\
& | & \textit{ctor} \textit{ty}_1 \dots \textit{ty}_n
\end{array}$$

$$\begin{array}{lcl}
\textit{predicates}, \textit{preds}, \bar{\pi} & ::= & \\
& | & \emptyset \\
& | & \textit{pred} \\
& | & \textit{pred}, \bar{\pi} \\
& | & \bar{\pi}_1, \bar{\pi}_2
\end{array}$$

$$\begin{array}{lcl}
\textit{scheme}, \textit{sch}, \sigma & ::= & \text{scheme} \\
& | & \forall \textit{tv}_1 \dots \textit{tv}_n. \bar{\pi} \Rightarrow \textit{ty} \\
& | & (\bar{\alpha}. \bar{\pi} \Rightarrow \textit{ty}) \quad \text{from tvset} \\
& | & \textit{ty} \quad \text{Monotype}
\end{array}$$

$$\begin{array}{lcl}
\textit{context}, \Gamma & ::= & \text{typing environment} \\
& | & \emptyset \\
& | & \Gamma, x : \textit{sch} \\
& | & x : \textit{sch}
\end{array}$$

$constraint, c$	$::=$	constraint
	$ty_1 \sim ty_2$	equality constraint
	$pred_{evidx}$	predicate constraint with evidence
$constraints, C$	$::=$	constraint set
	$\emptyset$	
	c	
	C_1, C_2	set join
$subst, \theta$	$::=$	substitution
	$\emptyset$	
	$tv \mapsto ty$	
	$\theta_1 \circ \theta_2$	composition
$evidence, ev$	$::=$	evidence term
	x	evidence variable
	$ctor\ ev_1 \dots ev_n$	dictionary constructor
$evctx, \Delta$	$::=$	evidence context
	$\emptyset$	
	$\Delta, evidx : pred$	

3.2 Judgments & Inference rules

We now, present the judgments as well as the inference rule. Each judgement comes with a remark that informally explains the implementation.

Unification computes a most general unifier (MGU) for equality constraints. The unification engine

- checks occurrence to prevent infinite types as shown in [section 2.4](#)
- requires constructors to match exactly
- unifies KApp with concrete type applications TApp, provided arities match.
- α -renames schemes before unifying them structurally.

$\theta = \text{unify}(ty_1, ty_2)$ (Unification of types)

$\frac{\text{U-REFL}}{\emptyset = \text{unify}(ty, ty)}$	$\frac{\text{U-VARL} \quad tv \notin \text{ftv}(ty)}{tv \mapsto ty = \text{unify}(tv, ty)}$	$\frac{\text{U-VARR} \quad tv \notin \text{ftv}(ty)}{tv \mapsto ty = \text{unify}(ty, tv)}$
$\frac{\text{U-ARROW} \quad \begin{array}{l} \theta_1 = \text{unify}(a, a') \\ \theta_2 = \text{unify}([\theta_1]b, [\theta_1]b') \end{array}}{\theta_2 \circ \theta_1 = \text{unify}(a \rightarrow b, a' \rightarrow b')}$	$\frac{\text{U-PROD} \quad \begin{array}{l} \theta_1 = \text{unify}(a, a') \\ \theta_2 = \text{unify}([\theta_1]b, [\theta_1]b') \end{array}}{\theta_2 \circ \theta_1 = \text{unify}(a \times b, a' \times b')}$	

Constraint Solving are done left-to-right, threading the substitution through. Predicate constraints are collected and returned as residuals for instance resolution.

$$\boxed{(\theta, \bar{\pi}) = \text{solve}(C)} \quad (\text{Constraint solving})$$

$$\begin{array}{c} \text{SLV-EMPTY} \\ \hline (\emptyset, \emptyset) = \text{solve}(\emptyset) \end{array} \quad \begin{array}{c} \text{SLV-EQ} \\ \theta = \text{unify}(ty_1, ty_2) \\ (\theta', \bar{\pi}) = \text{solve}([\theta]C) \\ \hline (\theta' \circ \theta, \bar{\pi}) = \text{solve}(ty_1 \sim ty_2, C) \end{array} \quad \begin{array}{c} \text{SLV-PRED} \\ (\theta, \bar{\pi}) = \text{solve}(C) \\ \hline (\theta, [\theta]pred, \bar{\pi}) = \text{solve}(pred_{\text{evidx}}, C) \end{array}$$

Instantiation replaces bound TVs with fresh TVs and returns the predicate constraints that must be satisfied.

$$\boxed{(ty, \bar{\pi}) = \text{inst}(sch)} \quad (\text{Scheme instantiation})$$

$$\begin{array}{c} \text{INST-FORALL} \\ tv'_1 = \text{fresh} \quad \dots \quad tv'_n = \text{fresh} \\ \theta = \{tv_1 \mapsto tv'_1, \dots, tv_n \mapsto tv'_n\} \\ \hline ([\theta]ty, [\theta]\bar{\pi}) = \text{inst}(\forall tv_1 \dots tv_n. \bar{\pi} \Rightarrow ty) \end{array}$$

Generalization abstract over TVs not free in the environment. Predicates mentioning such TVs become part of the context, that is,

A predicate is kept if and only if it mentions one or more qTVs³³ that also appears in the result type.

$$\boxed{sch = \text{gen}(\Gamma, ty, \bar{\pi})} \quad (\text{Generalization})$$

$$\begin{array}{c} \text{GEN-GEN} \\ \bar{\alpha} = (\text{ftv}(ty) \cup \text{ftv}(\bar{\pi})) \setminus \text{ftv}(\Gamma) \\ \hline (\bar{\alpha}. \bar{\pi} \Rightarrow ty) = \text{gen}(\Gamma, ty, \bar{\pi}) \end{array}$$

Expression Typing generates constraints that are used in constraint solving to determine³⁴ the actual type. The judgment $\Gamma \vdash e \Rightarrow ty; C$ reads:

Under environment Γ , expression e has type ty generating constraints C .

Based on the core calculus, inference applies to:

Variables by looking up in Γ and instantiate the scheme,

Constants which already have concrete types,

Lambda abstraction of which parameter³⁵ gets fresh TVs,

Application by generating constraints that make the function type match,

Product by generating constraints that make the product type match,

³⁴by applying all substitutions

³⁵a lambda abstraction only takes one (1) parameter, as mentioned before.

Conditional by requiring then/else-branch to agree, condition to be Bool.

Let by inferring body *after* inferring/solving/generalizing RHS,

Let rec by extending Γ with fresh TVs for all binders *before* inferring each RHS, whose generated constraints equate these with inferred types,

Fix where **fix** $\lambda f. e$ has type of e ,

Ascription by checking that inferred type is at least as general, which, takes different approach based on ranks where

- for rank-1 ascriptions, we simply unify.
- for rank- n (where $n \geq 2$, that is, if $TSch$ contains nested quantifiers), we have the following steps:
 1. **Instantiation** instantiate the scheme $\sigma = \forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau$ with fresh type variables $\bar{\beta}$, yielding τ_{inst} .
 2. **Constraint solving** solve the current constraints augmented with $\tau_e \sim \tau_{\text{inst}}$ to obtain a substitution θ' .³⁶
 3. **Rigidity check** For each binding $\alpha \mapsto \tau'$ in the substitutions θ' , if α is in the current rigid set $\mathcal{R}$, then τ' must equal α (from the property of rigids. i.e. it must *not* be specialized). Failing this check results in the ascription being rejected with a unification error.
 4. **Skolemization** As stated above and below, ensures the generality does match the declared (polymorphic) type.

Pattern matching where for

- each branch i : Patterns $p_{i,1}, \dots, p_{i,m}$ are checked against the corresponding discriminant types. Γ is then enriched with pattern bindings. TVs arising from patterns are marked as *rigid* (within their branches) which prevents leaking³⁷ across branches. RHS is inferred after, requiring all branches to agree (through constraints).
- discriminants: Each one e_j is inferred independently, yielding types $\tau_1, \dots, \tau_m$. whose types must agree with pattern types.

Additionally, pattern exhaustiveness and redundancy are checked separately (via `Exhaustive.exhaustWitness`) but not reflected in the typing judgment. Non-exhaustive matches or redundant branches generate warnings but are not type errors. Note that the latter is immediately removed from the matrix.

$$\boxed{\Gamma \vdash e \Rightarrow ty; C} \quad (Constraint\text{-}based\ inference)$$

I-VAR

$$\frac{x : sch \in \Gamma \quad (ty, \bar{\pi}) = \text{inst}(sch)}{\Gamma \vdash x \Rightarrow ty; C}$$

I-INT

$$\Gamma \vdash \text{intlitt} \Rightarrow \text{Int}; \emptyset$$

I-STRING

$$\Gamma \vdash \text{strlitt} \Rightarrow \text{String}; \emptyset$$

I-TRUE

$$\Gamma \vdash \text{true} \Rightarrow \text{Bool}; \emptyset$$

³⁶This can be seen as a premature solving. We *tentatively* solve them only to use them in the next step that is rigidity check.

³⁷this is mostly a precaution. The “leaking” described here is only likely to happen when pattern matching index families, which, is not supported, at all.

$\text{I-FALSE} \frac{}{\Gamma \vdash \mathbf{false} \Rightarrow \mathbf{Bool}; \emptyset}$	$\text{I-UNIT} \frac{}{\Gamma \vdash () \Rightarrow \mathbf{Unit}; \emptyset}$	$\text{I-LAM} \frac{tv = \mathbf{fresh} \quad \Gamma, x : tv \vdash e \Rightarrow ty; C}{\Gamma \vdash \lambda x. e \Rightarrow tv \rightarrow ty; C}$
$\text{I-APP} \frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2 \quad tv = \mathbf{fresh}}{\Gamma \vdash e_1 e_2 \Rightarrow a; (C_1, C_2), ty_1 \sim ty_2 \rightarrow a}$	$\text{I-PROD} \frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2}{\Gamma \vdash (e_1, e_2) \Rightarrow ty_1 \times ty_2; C_1, C_2}$	
$\text{I-COND} \frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad \Gamma \vdash e_2 \Rightarrow ty_2; C_2 \quad \Gamma \vdash e_3 \Rightarrow ty_3; C_3}{\Gamma \vdash \mathbf{if} e_1 \mathbf{then} e_2 \mathbf{else} e_3 \Rightarrow ty_2; (((C_1, C_2), C_3), ty_1 \sim \mathbf{Bool}), ty_2 \sim ty_3}$		
$\text{I-LET} \frac{\Gamma \vdash e_1 \Rightarrow ty_1; C_1 \quad (\theta, \bar{\pi}) = \mathbf{solve}(C_1) \quad sch = \mathbf{gen}(\Gamma, [\theta]ty_1, \bar{\pi}) \quad \Gamma, x : sch \vdash e_2 \Rightarrow ty_2; C_2}{\Gamma \vdash \mathbf{let} x = e_1 \mathbf{in} e_2 \Rightarrow ty_2; [\theta]C_2}$	$\text{I-FIX} \frac{tv = \mathbf{fresh} \quad \Gamma, f : tv \vdash e \Rightarrow ty; C}{\Gamma \vdash \mathbf{fix}(\lambda f. e) \Rightarrow ty; C, tv \sim ty}$	$\text{I-LETREC} \frac{tv = \mathbf{fresh} \quad \Gamma, x : (tv) \vdash e_1 \Rightarrow ty_1; C_1 \quad (\theta, \bar{\pi}) = \mathbf{solve}(C_1) \quad sch = \mathbf{gen}(\Gamma, [\theta]ty_1, \bar{\pi}) \quad \Gamma, x : sch \vdash e \Rightarrow ty; C}{\Gamma \vdash \mathbf{let} x = e_1 \mathbf{in} e \Rightarrow ty_1; [\theta]C}$

Since match clause and type ascription are not easy to correctly type in OTT and too convoluted in terms of readability, we manually type them as follows:³⁸

$$\begin{array}{c}
\Gamma \vdash e_j \Rightarrow \tau_j; C_j \quad (\forall j \in 1..m) \\
\alpha = \mathbf{fresh} \\
\text{For each branch } i \in 1..k : \quad |(\bar{p}_i)| = m \quad \Gamma \vdash p_{i,j} \Leftarrow \tau_j \Rightarrow \Gamma_{i,j}; C_{p_{i,j}} \quad (\forall j \in 1..m) \\
\Gamma_i = \Gamma_{i,m} \quad (\text{final extended environment for branch } i) \\
\bar{\beta}_i = \mathbf{ftv}(\text{bound variables in } \bar{p}_i) \quad \mathbf{pushRigid}(\bar{\beta}_i) \quad \Gamma_i \vdash r_i \Rightarrow \tau_{r_i}; C_{r_i} \quad \mathbf{popRigid}() \\
C_{\mathbf{unify}} = \{\tau_{r_i} \sim \alpha \mid i \in 1..k\} \quad C = \bigcup_{j=1}^m C_j \cup \bigcup_{i=1}^k \left(\bigcup_{j=1}^m C_{p_{i,j}} \cup C_{r_i} \right) \cup C_{\mathbf{unify}} \\
\hline
\Gamma \vdash \mathbf{match} e_1, \dots, e_m \mathbf{with} \mid p_{1,1}, \dots, p_{1,m} \rightarrow r_1 \Rightarrow \alpha; C \quad \text{I-MATCH}
\end{array}$$

$$\frac{\Gamma \vdash e \Rightarrow \tau_e; C_e}{\Gamma \vdash (e : \tau) \Rightarrow \tau; C_e, \tau_e \sim \tau} \quad \text{I-ASCRIBE-MONO}$$

$$\frac{\Gamma \vdash e \Rightarrow \tau_e; C_e \quad \sigma = \forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau \quad \bar{\beta} = \mathbf{fresh}^{|\bar{\alpha}|} \quad \theta = [\bar{\alpha} \mapsto \bar{\beta}] \quad \tau_{\mathbf{inst}} = \theta(\tau) \quad C_0 = \text{current constraints} \quad (\theta', _) = \mathbf{solve}(C_0, \tau_e \sim \tau_{\mathbf{inst}}) \quad \mathcal{R} = \text{current rigid set} \quad \forall (\alpha \mapsto \tau') \in \theta'. \alpha \in \mathcal{R} \implies \tau' = \alpha \quad \tau_{\mathbf{sk}} = \mathbf{skolem}(\sigma) \quad \text{add constraints for } \bar{\pi}}{\Gamma \vdash (e : \sigma) \Rightarrow \tau_{\mathbf{inst}}; C_e, \tau_e \sim \tau_{\mathbf{inst}}} \quad \text{I-ASCRIBE-POLY}$$

³⁸Readers shall note that because these rules are hand-typed, some symbols are not defined as a production rule above, or a formula in OTT, in this case, adequate natural language is used to define/describe them inline. The author has no one but himself to blame due to his unfamiliarity with OTT.

Patterns bind variables and generate constraints from the discriminants. The judgment $\Gamma \vdash pat \Rightarrow \Gamma'; C; ty$ reads:

Pattern pat against type ty extends Γ to Γ' , generating constraints C .

$\boxed{\Gamma \vdash pat \Rightarrow \Gamma'; C; ty}$			(Pattern inference)
P-WILD $\frac{}{\Gamma \vdash _ \Rightarrow \Gamma; \emptyset; ty}$	P-VAR $\frac{x \notin \text{dom}(\Gamma)}{\Gamma \vdash x \Rightarrow \Gamma, x : ty; \emptyset; ty}$	P-CONSTINT $\frac{}{\Gamma \vdash \text{intlit} \Rightarrow \Gamma; ty \sim \text{Int}; ty}$	
P-CONSTSTR $\frac{}{\Gamma \vdash \text{strlit} \Rightarrow \Gamma; ty \sim \text{String}; ty}$	P-CONSTTRUE $\frac{}{\Gamma \vdash \text{true} \Rightarrow \Gamma; ty \sim \text{Bool}; ty}$	P-CONSTFALSE $\frac{}{\Gamma \vdash \text{false} \Rightarrow \Gamma; ty \sim \text{Bool}; ty}$	
P-CONSTUNIT $\frac{}{\Gamma \vdash () \Rightarrow \Gamma; ty \sim \text{Unit}; ty}$	P-PROD $\frac{\begin{array}{l} a = \text{fresh} \quad b = \text{fresh} \\ \Gamma \vdash pat_1 \Rightarrow \Gamma_1; C_1; a \\ \Gamma_1 \vdash pat_2 \Rightarrow \Gamma_2; C_2; b \end{array}}{\Gamma \vdash (pat_1, pat_2) \Rightarrow \Gamma_2; (C_1, C_2), ty \sim a \times b; ty}$		

Since constructor applications are not easy to correctly type in OTT and too convoluted in terms of readability, we manually type them as follows:

$$\frac{\begin{array}{l} ctor : \sigma \in \Gamma \quad (\tau_{ctor}, \bar{\pi}) = \text{inst}(\sigma) \quad \text{peel}(\tau_{ctor}) = (\tau_1, \dots, \tau_n, \tau_{\text{res}}) \quad n = |\bar{p}| \\ \Gamma_0 = \Gamma \quad \Gamma_{i-1} \vdash p_i \Leftarrow \tau_i \Rightarrow \Gamma_i; C_i \quad (\forall i \in 1..n) \quad C_\pi = \bar{\pi} \end{array}}{\Gamma \vdash ctor \ p_1 \dots p_n \Leftarrow \tau \Rightarrow \Gamma_n; C_1, \dots, C_n, C_\pi, \tau \sim \tau_{\text{res}}} \text{ P-CTOR}$$

Where peel decomposes an arrow type into its argument types and result type:

$$\text{peel}(\tau_1 \rightarrow \tau_2 \rightarrow \dots \rightarrow \tau_n \rightarrow \tau_r) = (\tau_1, \tau_2, \dots, \tau_n, \tau_r)$$

The constructor scheme σ is looked up in Γ and instantiated with fresh type variables. Predicate constraints from instantiation are added to ensure typeclass requirements are propagated.

Below, we list some remarks for concepts mentioned above that are not properly introduced.

Remark (Skolemization). replaces TVs with skolems that cannot unify with anything except themselves.

$$\text{skolem}(\forall \bar{\alpha}. \bar{\pi} \Rightarrow \tau) = [\bar{\alpha} \mapsto \overline{\text{sk}_{\bar{\alpha}}}] (\tau)$$

where each sk_{α} is a fresh skolem constant (implemented as a special type constructor, e.g., $\text{?sk}.\alpha$ in the implementation).

The word *rigid* is to capture the key nature of skolems: $\text{sk}_{\alpha} \sim \tau$ succeeds iff $\tau = \text{sk}_{\alpha}$.

Remark (Rigid Set/Stack). The type system maintains a stack of rigid type variable sets to handle nested scopes correctly.

$$\begin{array}{ll} \text{pushRigid}(\bar{\alpha}) : & \mathcal{R} := \mathcal{R} \cup \bar{\alpha}; \quad \text{stack} := \bar{\alpha} :: \text{stack} \\ \text{popRigid}() : & \text{let } (S :: \text{rest}) = \text{stack} \text{ in } \mathcal{R} := \mathcal{R} \setminus S; \quad \text{stack} := \text{rest} \end{array}$$

Invariant. At any point during inference, the rigid set maintains the fact that

$$\mathcal{R} \equiv \bigcup \text{stack}$$

(i.e. it is the union of all sets currently on the stack.) When solving constraints, any substitution $\alpha \mapsto \tau$ where $\alpha \in \mathcal{R}$ and $\tau \neq \alpha$ is rejected.

Note that the $(:=)$ operation is implemented by the ST monad, through the `MonadStateOf` interface.

Remark (Solving Rigids). Though not shown above in the inference rule, the constraint solver respect rigid TVs:

$$\frac{\alpha \in \mathcal{R} \quad \tau \neq \alpha}{\text{solve}(\alpha \sim \tau) = \mathbf{error}} \text{SOLVE-RIGID} \qquad \frac{\alpha \in \mathcal{R}}{\text{solve}(\alpha \sim \alpha) = \emptyset} \text{SOLVE-RIGID-OK}$$

Remark (Soundness). The type system should maintain soundness through several mechanisms:

Skolems During pattern matching and typing ascription, some TVs are marked as *rigid* and tracked in rTV . Substitutions that would specialize them to incompatible types are rejected by the solver. This is thought to prevent unsound generalizations.

Rank-N skolemization When checking $e : \forall tv.\pi \Rightarrow ty$, we replace tv with a fresh skolem constant (in order to make it truly general) to ensure expressions do work for all instantiations.

Occurs check is used to prevent users from constructing equirecursive types such as $\alpha \sim \alpha \rightarrow \beta$ or $\alpha \sim \alpha \times \beta$ arising from codes such as `fun x  $\Rightarrow$  x x` or `fun | (x, y)  $\Rightarrow$  x | z  $\Rightarrow$  z`, respectively.

Hygienic generalization Only TVs not free in Γ are generalized. Predicates are kept based on the condition described above that shall avoid vacuous constraints.

Syntax restrictions Lastly, if one looks closely at the BNF grammar of type expression or the formal grammar presented above, they will find that, it is actually not possible to write $(\forall a.a \rightarrow a) -> \text{Int} \times \text{Bool}$ as though MLType embeds scheme, the parsing facility does not admit schemes inside monotypes. Indeed, the only way to introduce embedded scheme is through polymorphic fields in datatype declarations, which, is not harmful. This is perhaps, the strongest restriction that is thought to guarantee the soundness of the type system.

Remark (Counterexamples: The need of rigids). Below we provide some counterexamples that emphasize the need of rigids:

- **(Wrong Specialization)** Consider the following

```
let id = (fun x  $\Rightarrow$  x : forall a, a -> a) 1

-- [ERROR] Can't unify type Int with ?sk.a.
let id'   : forall a, a -> a = fun x  $\Rightarrow$  (x : Int)
let id'' x : forall a, a -> a = x + 1
```

The first example is acceptable: The ascription claims polymorphism, but the body is monomorphic, which is fine since we are instantiating the polymorphic function.

In the second example however, It would be unsound for a , introduced as a rigid inside `fun x  $\Rightarrow$  ...` to be specialized to `Int`. Same applies to the third.

- **(Polymorphism recovery)** Typically, typeclass methods (which is very likely to be polymorphic themselves) are used directly which allows the special treatment of them like in Haskell '98. However, one may wish to write:


```

type Functor (f : 1) = Functor (forall a b, (a -> b) -> f a -> f b)

let prog (Functor f) =
  let l = 1 :: 2 :: 3 :: Nil in
  and l' = "1" :: "2" :: "3" :: Nil in
  and const k _ = k in
  let x = f (_ + 1) l in
  and y = f (const ()) l'
  in (x, y)

```

This is perfectly acceptable, yet, do note the importance of rigids: a and b are rigid within each use, within each call. This allows the independent instantiation of them as f is specialized to different concrete type at x ($a \mapsto \text{Int}, b \mapsto \text{Int}$) and y ($a \mapsto \text{String}, b \mapsto \text{Unit}$).

3.3 Instance resolution

Merely associating instances with typeclasses is not enough at all: We, or, to be precise, the compiler, requires a general procedure (algorithm) to lookup the right instance for the right type of some class.³⁹ This process is called *instance resolution*. The algorithm described below searches the instance environment for a matching instance, recursively resolving any context constraints.

Instance metadata Each registered instance I 's metadata is a structure

$$I = \langle \text{iname}, \text{cls}, \bar{\alpha}, \bar{\pi}_{\text{ctx}} \rangle$$

where

- `iname` is the instance's unique identifier (naming convention: `i_Eq_0`),
- `cls` is the class head,
- $\bar{\alpha}$ is the list of type arguments in the instance head,
- $\bar{\pi}_{\text{ctx}}$ is the list of context predicates.

e.g. an instance declaration `instance : forall  $\alpha$  [Eq  $\alpha$ ], Eq (List  $\alpha$ ) = ...` produces

$$I = \langle \text{i_Eq_1}, \text{Eq}, [\text{List } \alpha], [\text{Eq } \alpha] \rangle$$

Head Unification refers to the procedure of checking whether a goal $C \bar{\tau}$ can match an instance head $C \bar{\alpha}$ before attempting full resolution.

³⁹Or as in PL jargon: finding a *dictionary* (evidence term) that *witnesses* a predicate constraint $\pi = C \bar{\tau}$.

Algorithm 1 Head Unification

```

1: function UNIFYHEAD( $\bar{\tau}_{\text{goal}}, \bar{\alpha}_{\text{inst}}$ )
2:   if  $|\bar{\tau}_{\text{goal}}| \neq |\bar{\alpha}_{\text{inst}}|$  then
3:     return error
4:   end if
5:    $\theta \leftarrow \emptyset$ 
6:   for  $i \leftarrow 1$  to  $|\bar{\tau}_{\text{goal}}|$  do
7:      $\theta' \leftarrow \text{unify}(\text{apply}(\theta, \tau_i), \text{apply}(\theta, \alpha_i))$ 
8:      $\theta \leftarrow \theta' \circ \theta$ 
9:   end for
10:  return  $\theta$ 
11: end function

```

Resolution Algorithm The main resolution algorithm maintains a *visited set* $\mathcal{V}$ to detect cycles (which indicates either ambiguous or divergent instance resolution).

Algorithm 2 Instance Resolution

```

1: function RESOLVE( $\Gamma, \pi, \mathcal{V}$ )
2:   Input: Environment  $\Gamma$ , predicate  $\pi = C \bar{\tau}$ , visited set  $\mathcal{V}$ 
3:   Output: Evidence expression  $e$  s.t.  $e : C \bar{\tau}$ , or error
4:
5:   if  $\pi \in \mathcal{V}$  then
6:     return error: “cyclic instance resolution”
7:   end if
8:    $\mathcal{V} \leftarrow \mathcal{V} \cup \{\pi\}$ 
9:
10:   $insts \leftarrow \Gamma.instInfo[C]$  ▷ All instances for class  $C$ 
11:  if  $insts = \emptyset$  then
12:    return error: “no instance for  $\pi$ ”
13:  end if
14:
15:   $found \leftarrow \text{none}$ 
16:  for each  $I = \langle iname, C, \bar{\alpha}, \bar{\pi}_{ctx} \rangle$  in  $insts$  do
17:    try:
18:       $\theta \leftarrow \text{UNIFYHEAD}(\bar{\tau}, \bar{\alpha})$ 
19:       $\bar{\pi}'_{ctx} \leftarrow \text{apply}(\theta, \bar{\pi}_{ctx})$ 
20:      for each  $\pi' \in \bar{\pi}'_{ctx}$  do
21:         $\_ \leftarrow \text{RESOLVE}(\Gamma, \pi', \mathcal{V})$  ▷ Ensure solvable
22:      end for
23:       $e \leftarrow (iname : C \bar{\tau})$  ▷ Ascribed instance reference
24:      if  $found \neq \text{none}$  then
25:        return error: “ambiguous instance for  $\pi$ ”
26:      end if
27:       $found \leftarrow \text{some}(e)$ 
28:    catch: continue ▷ Try next instance
29:  end for
30:
31:  if  $found = \text{none}$  then
32:    return error: “no matching instance for  $\pi$ ”
33:  end if
34:  return  $found$ 
35: end function

```

Remark (ResolveM). The algorithm described above runs inside **ResolveM** monad. Just like any monad we previously mentioned, it is a ST backed state monad with error handling (hence EST), whose state is `ResolveState`, which is precisely the visited set $\mathcal{V}$.

```

abbrev ResolveState := Std.HashSet Pred
abbrev ResolveM  $\sigma$  := StateRefT ResolveState (EST TypingError  $\sigma$ )

variable { $\sigma$  : Type}

instance : MonadLift (Except TypingError) (EST TypingError  $\sigma$ ) where
  monadLift
  | .error e  $\Rightarrow$  throw e
  | .ok e  $\Rightarrow$  return e

```

Meanwhile, the `MonadLift` instance defines a natural transformation from `Except` to `EST`, lifting fail-able but pure actions to run in it.

Remark (Properties of instance resolution). The key design points of this algorithm are described below:

Termination has not been proved, as noted by the partial modifier in the source. Intuitively, It terminates if the dependency graph of a instance is well-founded i.e. does not contain infinite chains of instance dependencies. The set $\mathcal{V}$ (line 5) detects direct cycles, but may not (or does not) guarantee termination for all pathological instances.

Semantic coherence is enforced through a *single-match* policy: If multiple instances could match a goal (line 24), resolution fails with an ambiguity error, which, guarantees that the meaning of a program does not depend on which instance is arbitrarily chosen. However, users should note that, contrary to the usual practice, this restriction applies to all instances regardless of their generality, that is, the more specific one would not be chosen although it may make more sense doing so, and will result in a compile-time error. To allow the existence of multiple matched instances, one way is to introduce a precedence and a default rule like the above practice, but this currently is not supported.

Evidence generation refers to ascribed instance identifiers:

$$e = (\text{iname} : C \ \bar{\tau})$$

Which is later *elaborated* into dictionary-passing style during the System F IR translation phase from which is then encoded in the field projection node `Proj src mname i ty`.

Context propagation refers to instantiating context predicates with the unifying substitution (line 19) and the recursively resolving of them (line 20), from instances with context predicates $\bar{\pi}_{\text{ctx}}$ (e.g., `Eq a in instance : forall  $\alpha$  [Eq  $\alpha$ ], Eq (List  $\alpha$ ) = ...`)

We shall simulate this algorithm with a simple example below: Consider resolving `Eq (List Int)` with instances:

$$\begin{aligned} I_0 &= \langle \text{i_Eq_0}, \text{Eq}, [\text{Int}], \emptyset \rangle, \\ I_1 &= \langle \text{i_Eq_1}, \text{Eq}, [\text{List } \alpha], [\text{Eq } \alpha] \rangle. \end{aligned}$$

1. Goal: `Eq (List Int)`
2. Try I_0 : `unify(List Int, Int)` fails
3. Try I_1 : `unify(List Int, List  $\alpha$ )` succeeds with $\theta = [\alpha \mapsto \text{Int}]$
4. Context: `apply( $\theta$ , [Eq  $\alpha$ ]) = [Eq Int]`
5. Recursively resolve `Eq Int`:
 - (a) Try I_0 : `unify(Int, Int)` succeeds with $\theta = \emptyset$
 - (b) Context: $\emptyset$, no recursion needed
 - (c) Return `(i_Eq_0 : Eq Int)`
6. Return `(i_Eq_1 : Eq (List Int))`

Remark (Comparison with GHC). GHC implements a more robust, sophisticated typeclass facility. Although *Tigris* has learned and taken many inspirations from it, its instance resolution strategy is much simpler than GHC's:

- Tigris does not allow overlapping instances, regardless of any ordering, whereas GHC provides precedence that controls the granularity of instance resolution;
- Tigris does not have instance chains as there is no mechanism for specifying fallback instances or instance priority

However, when compared to Haskell '98 or 2010, *Tigris*' instance declarations is somewhat more expressive as we have some form of FlexibleContexts⁴⁰ as it is possible to define **instance** $\forall a$ [Eq a], Eq ($a * a$) = ..; Additionally, multi-parameter classes are also supported, such as the HAdd and HEq class mentioned above.

3.4 System F elaboration

The typed syntax TExpr, after instance resolution, is elaborated to FExpr. Below, we discuss an informal description of the elaboration.

Given a scheme

$$\forall \alpha_1 \cdots \alpha_n [P_1, \dots, P_k], \tau$$

we elaborate a binding rhs to

$$\Lambda \alpha_1, \dots, \alpha_n. \lambda d_1, \dots, d_k. \text{body}$$

where $d_1, \dots, d_k$ are the respective instances of preds $P_1, \dots, P_k$. Except when the rhs already starts with the exact k dictionary lambdas, that is, wrappers auto-generated for class methods, in which case we only add TyLams.

At each Var occurrence:

$$\text{Var } x : \sigma \text{ where } \sigma = \forall \bar{\alpha} \bar{P}, \text{tbody}$$

is elaborated to

$$\begin{aligned} & \text{TyApp } \cdots (\text{TyApp } (\text{Var } x_{\text{ty}_{\text{mono}}}) \alpha_1) \cdots \alpha_n \\ & \text{App}(\cdots) d_1 \\ & \vdots \\ & \text{App}(\cdots) d_k \end{aligned}$$

A dictionary argument⁴¹ is either

- in scope, that is, matching one of the predicate templates introduced by lambdas, or,
- synthesized by instance resolution, producing an untyped expression. Inference and elaboration is then re-run on that expression to transform it to an FExpr. This procedure is recursive.

In case if a method field type is polymorphic (e.g. Functor), as in TSch (Forall as [] τ), it is reified as a System F type abstractions by wrapping the projected field with TyLam, for each binder in as. This is thought to be canonical for Higher-ranked types. Do note, however, per-method predicate contexts inside **TSch** are currently unsupported and will be rejected.

⁴⁰An extension of GHC that lifts the Haskell 98 restriction that the type-class constraints (anywhere they appear) must have the form class type-variable or class (type-variable type1 type2 .. typen).

⁴¹named $d_P.\text{cls_}i$ where P is the predicate, i is the instance counter for that class; a case it is memoized, the identifier is prepend with a r .

Dictionary memoization refers to the memoization of synthesized dictionaries. It is common for a single instance to be used multiple times, in which case it becomes rather inefficient, both compile-time and runtime as each encounter results in a newly synthesized dictionary, or a dictionary application of multiple dependent dictionaries (as is the case for instances with contexts). Memoization tracks the first usage of a dictionary at function-level, upon encountering multiple usage, the synthesized instance is reused, and the function body is prepended by a let-exp that binds this instance that *hoist* the inlined dictionary application. This is similar to common subexpression elimination, except in this case it is targeted and not general at all, which comes with the benefit of being more convenient to implement. Consider the hierarchy of Functor, Applicative and Monad, in a rather elaborated example:

```
class Applicative (f : 1) =
  { pure : ∀ a, a -> f a
    -- lazy to preserve eval semantics
    , seq : ∀ a b, f (a -> b) -> (Unit -> f a) -> f b }
class Monad (m : 1) =
  { bind : ∀ a b, m a -> (a -> m b) -> m b
    , return : ∀ a, a -> m a }

infixl 55 ">=>" => bind
instance ∀(m : 1) [Monad m], Applicative m =
  { pure = return
    , seq f x = f >=> fun f => x () >=> fun x => return $ f x
    }
instance ∀(f : 1) [Applicative f], Functor f =
  { map f x = seq (pure f) fun _ => x }

instance Monad Option =
  { return = Some
    , bind
      | None, _ => None
      | Some x, f => f x }

let seqOp = seq (Some id)
```

With `i_Monad_0`, `i_Applicative_0` corresponding to the `Option` instance of `Monad` and the general `Applicative` instance, the System F IR result is

```
let seqOp : ∀ α, (Unit → Option α) → Option α =
  let rd_Applicative_1 : Applicative Option =
    let rd_Monad_0 : Monad Option = i_Monad_0
    in i_Applicative_0@Option rd_Monad_0
  in (λ α. rd_Applicative_1[1, seq]@Option (Some@(α → α) id@α))
```

As shown above, memoization have come into effect as the application of dictionary is cached and is bound by `rd_Applicative_1`, which, has a counter of 1, this indicates that dictionary application creates (synthesizes) new dictionaries. Therefore it is much necessary to have memoization as it mainly boosts compile performance.

In practice, any elaboration runs inside the `F` monad, which, together with recursion state⁴², is defined to be

⁴²It is worth noting that elaboration does not only depend on the state shown below, additional trivial environments are not shown here, please refer to `fsyntax.lean` and `fexpr.lean`.

```

structure FState where
  log      : Logger := ""
  memo     : Std.HashMap Pred (String × Scheme × FExpr) := ∅
  gensym   : Nat := 0
  deriving Inhabited

```

```

abbrev F := EStateM TypingError FState

```

A canonical state monad equipped with error reporting is used here, contrary to before: This is because of the many usage of local states in recursive call during elaboration. Stack-allocated (function parameters) states are much easier to work with in this case, than a *real* threadlocal mutable state.

Remark (SysF IR pretty-printed syntax). Here we have shown a more elaborated example of pretty-printed System F IR source. It should be obvious for anyone that is familiar with OCaml or Haskell, especially the type application syntax $\langle \text{term} \rangle @ \langle \text{type-expr} \rangle$. But for reference, we present the relevant pretty-printer implementation below:

```

1  local instance : Std.ToFormat Pattern where
2    format := .text [] Pattern.toString
3  open Std Std.Format in
4  partial def FExpr.unexpand : FExpr → Format
5    | .CI i _ | .CB i _ | .CS i _ ⇒ repr i
6    | .CUnit _ ⇒ format ()
7    | .App f a _ ⇒ parenL? f ++ group (indentD (parenR? a))
8    | .Proj src mname i _ ⇒ parenL? src ++ sbracket (format i ++ ", " ++ format mname)
9    | .Cond c t e _ ⇒
10     "if" ◇ unexpand c ↔ "then" ++ indentD (unexpand t)
11     ↔ "else" ++ indentD (unexpand e)
12    | .Fix e _ ⇒ "rec" ◇ unexpand e
13    | .Var x _ ⇒ format x
14    | .Prod' p q _ ⇒ paren $ unexpand p ◇ ", " ◇ unexpand q
15    | .Fun param pTy body _ ⇒
16     group $ "fun" ◇ param ◇ ":" ◇ pTy.toString ◇ "⇒" ++ group (indentD (unexpand body))
17    | .Match discr branches .. ⇒
18     let discr := discr.map unexpand
19     let br := branches.map fun (pats, e) ⇒
20       "|" ◇ joinSep' pats ", " ◇ "⇒" ++ group (indentD (unexpand e))
21     group $ "match" ◇ joinSep' discr ", " ◇ "with" ↔ joinSep' br line
22    | .Let binds body _ ⇒
23     let (binds, recflag) := binds.foldl (init := ([], false)) fun (acc, recflag) (id, sch, fe) ⇒
24       match fe with
25       | .Fix (.Fun _ _ body _) _ ⇒
26         (acc.push $ id ◇ ":" ◇ toString sch ◇ group ("=" ++ indentD (unexpand body)), true)
27       | _ ⇒
28         (acc.push $ id ◇ ":" ◇ toString sch ◇ group ("=" ++ indentD (unexpand fe)), recflag)
29     let recStr := if recflag then .text " rec " else .text " "
30     "let" ++ recStr ++ joinSep' binds (line ++ "and ") ↔ "in" ◇ unexpand body
31    | .TyLam a body _ ⇒
32     let (tvs, body) := Helper.peelTyLam [a.elimStr] body
33     group $ paren $ "λ" ◇ joinSep tvs " " ++ "." ++ indentD (unexpand body)
34    | .TyApp f ty ⇒ parenL? f ++ "@" ++ parenT ty
35  where

```

```

36 parenR?
37   | p@(.App ..) | p@(.Fun ..) | p@(.Cond ..) | p@(.Let ..) | p@(.Match ..) ⇒ paren (unexpand p)
38   | p ⇒ unexpand p
39 parenL?
40   | p@(.Fun ..) | p@(.Cond ..) | p@(.Let ..) | p@(.Match ..) ⇒ paren (unexpand p)
41   | p ⇒ unexpand p
42 parent
43   | t@(TVar _) | t@(TCon _) | t@(TApp _ []) | t@(KApp _ []) ⇒ text t.toString
44   | t ⇒ paren t.toString

```

Remark (On the usefulness of System F IR). Since the beginning of this project/thesis, the author himself has a very clear goal of compiling *Tigris* to Common Lisp (SBCL). Since LISP is a dynamically typed language with some form of type inference and propagation (SBCL-only, probably not gradual typing), There is not much need to do specialization of generics, or to preserve typing information for next-stage lowering.⁴³

Because of this, we *did not* actually need a proper system F IR, despite we did, as typeclass elaboration is perfectly doable without it. This is where the author has displayed both his long-term thinking ability and his shortsightedness:

- The former, as in for later proper implementation of arbitrary rank polymorphism and/or inductive families, a System F IR is useful to have;
- The latter, as in after Lambda IR, typing information is gone since it does not preserve types, leaving codegen harder, and most importantly, *much less robust*. Where we have to record the *shapes* of symbols⁴⁴ and to do some form of pattern matching on IR to temporarily fix a bug. Moreover, it is much less feasible if we were to develop some other backends that need types (e.g. C).

Although this does not have much to do with the usefulness of SysF IR, the insight is the same:

It is crucial to have a properly typed IR throughout compilation of programs.

4 The *Lambda* IR, CPS, Codegen & FFI

Elaboration to System F IR marks the end of “going up”, whereas *Lambda* IR⁴⁵ serves as an actual intermediate language that marks the beginning of code lowering. *Lambda* IR should be classified as in A-normal form, or ANF for short, a type of IR frequently used in functional programming language compilers.

The essence of ANF is *names*. Object, or any (nontrivial) expression, is referred by name, by immediately capture them in a let-bound variable. This, to some form, mimics the assembly code of the same program. Although *Tigris* compiles to a high-level language, its form is not what normal programming styles looks like and thus SBCL may benefit from it, as the *Tigris* compiler is doing some of the jobs for it.

⁴³though it has been painfully proven to be convenient, through coding practice.

⁴⁴which itself, is not perfectly reliable.

⁴⁵The name *Lambda* is a direct reference to the IR Appel used in his *Compiling with Continuations* and presumably used in the SML/NJ compiler. *Tigris* itself is a nod to Appel’s *Modern Compiler Implementation in ML*, of which the cover features a tiger. *Tigris* is latin.

4.1 The *Lambda* IR

Definition of *Lambda* IR Design of *Lambda* IR focuses on five aspects: Value, Rhs, Stmt, Tail and LExpr. We shall present the definition of these parts individually, together with a brief explanation of their intents. Readers shall note that these definitions are mutually recursive.

Value Also called immediate values. They cannot be further reduced.

```
inductive Value where
| var    (x : Name)
| cst    (k : Const)
| constr (tag : Name) (fields : Array Name)
| lam    (param : Name) (body : LExpr)  -- single-argument lambdas; multi-arg via tuples
deriving Repr, Inhabited
```

Note that *closures* (lambdas) at this stage are not yet converted and capture free variables.

Rhs denotes the right-hand sides of ANF where all arguments are variables

```
inductive Rhs where -- x : Rhs
| prim    (op : PrimOp) (args : Array Name)      -- x := op(args1, ..., argsn)
| proj    (src : Name) (idx : Nat)                -- x := srcidx
| mkPair   (a b : Name)                          -- x := ⟨a, b⟩
| mkConstr (tag : Name) (fields : Array Name)     -- x := tag⟦fields1, ..., fieldsn⟧
| isConstr (src : Name) (tag : Name) (arity : Nat) -- x := src matches ⟨tag/arity⟩
| call     (f : Name) (arg : Name)                -- x := f(arg)
deriving Repr, Inhabited
```

Stmt right now only has single let binding, which, in practice, is no difference from a letexp.

```
inductive Stmt where
| let1 (x : Name) (rhs : Rhs)
deriving Repr, Inhabited
```

Tail denotes the tail positions, which includes return, application, conditional and pattern matching branches. Tail is important in marking tail positions, which is utilized in later tailcall transformations.

```
inductive Tail where -- tl : Tail
| ret    (x : Name)                -- tl := RET x
| app    (f : Name) (arg : Name)    -- tl := f(arg)T
| cond   (condVar : Name) (tBranch eBranch : LExpr) -- tl := if cond then t else e
| switchConst (scrut : Name)        -- tl := caseC of casesi
    (cases : Array (Const × LExpr))
    (default? : Option LExpr)
| switchCtor (scrut : Name)          -- tl := case of casesi
    (cases : Array (Name × Nat × LExpr))
    (default? : Option LExpr)
| matchFail (pats : Array Name)      -- tl := MATCHFAIL(pats)
deriving Repr, Inhabited
```

Note that constant switches and constructor switches are treated differently – This is to facilitate later constant folding more convenient to implement.

MATCHFAIL is made a special token to denote the *fallback* branch of a partial match. As said before, entering such branch results in a nonrecoverable runtime error. The reason that it takes `pats`, an array of `Names` i.e. strings, is to provide the users with the symbols of the match discriminants, which, is directly passed onto codegen, printing from the LISP debugger.

`LExpr` is the main body of an program in *Lambda IR*.

```
inductive LExpr where
  /-- always ends in tails -/
  | seq    (binds : Array Stmt) (tail : Tail)
  | letVal (x : Name) (v : Value) (body : LExpr) -- let $\iota$   $x = v$  in body
  /-- Nested let Rhs bindings. -/
  | letRhs (x : Name) (rhs : Rhs) (body : LExpr) -- let  $x = rhs$  in body
  /-- Recursive function bindings. -/
  | letRec (funs : Array LFun) (body : LExpr)
    -- let $\omega$ 
    -- label  $fun_1 : \dots$ 
    -- label  $fun_2 : \dots$ 
    --  $\vdots$ 
    -- in body
```

deriving Repr, Inhabited, BEq

```
/-- A top-level function in the Lambda IR. -/
structure LFun where
  fid    : Name
  param  : Name
  body   : LExpr
deriving Repr, Inhabited
```

Here, normally what would be one `letexp` is divided into 3 sub-lets based on their RHS: a **let** ι that binds immediate value, a **let** that binds regular Rhs, and finally a **let** ω that binds a group of mutually recursive functions.

In addition to these mutually recursive definitions, we additionally have `LFun` that represents a top-level function. Readers with a keen heart will notice that `letRec` carries an array of `LFuns` – `LFun` at this position doesn't mean that `funs` have to be toplevel. We use `LFun` instead of `Name × Name × LExpr` is purely out of performance reasons: they have compatible representation, which, could save some potentially unnecessary conversions.

Sequence operations – `seq` – although *Tigris* is a pure language, we use `seq` to capture tails as it is the only constructor in `LExpr` that takes `Tails`. Also, someday may turn *Tigris* into a impure functional language like OCaml which, can be convenient if `seq` is present in `LExpr`.

Lambda IR is pretty-printed. In order for readers to get acquainted with the syntax, we'll omit the implementation of pretty printer, and instead, provide a concrete example of a simple evaluator:

```
type Expr a =
  | Atom a
  | Add (Expr a) (Expr a)
  | Sub (Expr a) (Expr a)
  | Mul (Expr a) (Expr a)
  | Div (Expr a) (Expr a)
```

```
let rec eval
```

```

| Atom a ⇒ a
| Add e1 e2 ⇒ eval e1 + eval e2
| Sub e1 e2 ⇒ eval e1 - eval e2
| Mul e1 e2 ⇒ eval e1 * eval e2
| Div e1 e2 ⇒ eval e1 / eval e2

let prog =
  Mul (Atom 20) $
    Sub (Mul (Atom 10)
          (Atom 20)) $
      Div (Atom 2400) $
        Add (Atom 120) $
          Add (Mul (Atom 10)
                (Atom 20)) $
            (Atom 0)
;;
let 3860 = eval prog

```

yields, in *Lambda IR*

```

main (arg) {
  letw
  label eval(args0):
    let _pL#?x0 = args0[0]
    case _pL#?x0 of
      «Add/2» →
        let p1 = _pL#?x0[0]
        let p2 = _pL#?x0[1]
        letu u3 = ()
        let pair4 = ⟨p1, u3⟩
        let call5 = eval(pair4)
        letu u6 = ()
        let pair7 = ⟨p2, u6⟩
        let call8 = eval(pair7)
        let p9 = ADD(call5, call8)
        RET p9
      (...)
      «Atom/1» →
        let p28 = _pL#?x0[0]
        RET p28
      (...)
in
  letu c38 = 20
  let con39 = Atom[[c38]]
  (...)
  letu c42 = 20
  let con43 = Atom[[c42]]
  let con44 = Mul[[con41, con43]]
  (...)
  letu c51 = 20
  let con52 = Atom[[c51]]

```

Primitive operation	Description
add	Integer addition
sub	Integer subtraction
mul	Integer multiplication
div	Integer division
eqInt	Integer equality
eqStr	String equality
eqBool	Bool equality

Table 6: Primitive operations of *Lambda IR*

```

let con53 = Mul[con50, con52]
(...)
let! u61 = ()
let pair62 = <con60, u61>
let call63 = eval(pair62)
let! c64 = 3860
let cmp65 = EQi(call63, c64)
if cmp65 then RET con60 else MATCHFAILURE
}

```

we shall mention some peculiar details during lowering, which itself, is implemented in CPS style: As stated above, having a system F is somewhat overkill since the compilation target being a dynamically typed language. To reduce noises, a system F IR is stripped of `TyLam`, `TyApp`.

Shortpath: Constructors & Primitive operations First, we list the seven primitive operations and nullary constructors that is not impacted by the additional overhead introduced by the function calling convention: [Table 6](#)

Detection of this operations is through pattern matching of the application of these primitives, hence it is sometimes necessary to η -expand them in, e.g., method registration which is seen before. In practice, primitives and normal functions are only differentiated by their names. That is, applications for primitives can also be done through `funapp` branch – However, it is rather inefficient to do so, which is why the `lowerF` function handles theses primitives (and nullary constructors) first:

```

partial def lowerF
  (e : FExpr) (p : Env)
  (ctors : Std.HashMap String Nat)
  (k : Name -> M σ LExpr) : M σ LExpr := do
  match matchBinaryPrim e with
  | some (op, x, y) =>
    lowerF x p ctors fun vx =>
      lowerF y p ctors fun vy => do
        let r <- fresh "p"
        let cont <- k r
        return .letRhs r (.prim op #[vx, vy]) cont
  | none =>
    match matchCtorApp ctors e with
    | some (cname, args, ar) =>
      if ar == 0 && args.isEmpty then
        let r <- fresh "con"

```

```

    let cont <- k r
    return (.letRhs r (.mkConstr cname #[]) cont)
  else lowerCtorApp cname args ar p ctors k
| none => (...) -- normal lowering branches.

```

Dictionary projections are nodes that are specialized for typeclass methods. Normally, methods are projected through the use of pattern matching, according to canonical dictionary passing implementations. This compilation overhead is reduced by the use of Projs, which, is threaded through *Lambda IR* lowering.

Function, function applicaton & uncurrying Proper functional languages enforces currying. However, it is inefficient to generate n functions for a n -arity function, which, is also harder for target languages to do tailrec optimizations, which, is why most functional language compilers do *uncurrying* – the action of *squashing* curried parameters into a single canonical products⁴⁶ that can be encoded by the `lam` constructor. The lowering of functions have several steps below:

1. **decomposeLamChain** is used to squash *consecutive* function⁴⁷ heads i.e. `fun ..`⁴⁸ which returns a 3-tuple $(p0, rest, core)$ where $p0$ refers to the first parameter, *rest* refers to the rest and *core* refers to the function body.
2. **η -expansion** not only refers to the practice in lambda calculus, in our case, it refers the corner case of applying a projected⁴⁹ function not properly covered by the `App` branch: Consider this example:

```

type BoxedAdd = Boxed (Int -> Int -> Int)
let boxedAdd = Boxed (_ + _)

let unwrapBoxed (Boxed f) = f
let (Boxed unboxedAdd') = boxedAdd
let main =
  ( unwrapBoxed boxedAdd 20 20
    , unboxedAdd' 30 30)

```

where `unwrapBoxed boxedAdd 20 20` is called a *higher-order application*, where, the application of `boxedAdd` at `main`. This, however, cannot be caught by `lowerF` as it only recognizes the application head `unwrapBoxed` and it sees no application inside `unwrapBoxed`, thus, this application would not be translated correctly by a naive interpreter. In this case, `unwrapBoxed` is implicitly η expanded (in spirit) as `let unwrapBoxed (Boxed f) x y = f x y`. As shown in the LISP object code below:

```

(defun |fn1001| (|payload| |k|) ; unwrapBoxed
  (declare (optimize (speed 3) (safety 0) (debug 0)) (ignorable |payload|))
  (let ((|a| (car |payload|)))
    (let ((|_pL#?x_0| (car |a|)))
      (let ((|_pR#?x_0| (cdr |a|)))
        (let ((|_pL#?η0| (car |_pR#?x_0|)))
          (let ((|_pR#?η0| (cdr |_pR#?x_0|)))

```

⁴⁶in our implementation, specifically.

⁴⁷i.e. `fun x => fun y => ...`

⁴⁸squashing is unconditional – even though several `fun` heads might be bounded by different names, they are squashed as well.

⁴⁹from pattern matching

```

(cond
  ((eq (car |_pL#?x0|) '|Boxed|)
    (let ((|p5| (svref (cdr |_pL#?x0|) 0)))
      (let ((|pair6| (cons |_pL#η0| |_pR#η0|)))
        (let ((|_code1002| (svref (cdr |p5|) 0)))
          (let ((|Γc1003| (svref (cdr |p5|) 1)))
            (let ((|pc1004| (cons |pair6| |Γc1003|)))
              (funcall (the function |_code1002|) |pc1004| |k|))))))))))

```

where $..ηN$ like $_{pR\#\eta 0}$ denotes a parameter produced by η -expansion.

One may wonder, how can we keep semantic consistency of CBV if we do η -expansion – Since some programs (e.g. Y combinator) would otherwise diverge. The answer is we (sort of) *can't*, which is why, η -expansion is only applied to the condition described above.

Besides, Lean does it as well and is also based on similar condition (type signature), See this [proofexchange post](https://proofexchange.stackexchange.com/a/4424) ⁵⁰ by me and this [github issue@leanprover/lean4#5908](https://github.com/leanprover/lean4/issues/5908) ⁵¹.

3. **Syntactic arity** refers to the user-defined function arity, which can introduce ambiguity when mixed with higher-order function/application as seen above. Normally, the lowering facility determines a function's arity based on its type signature, this, is proved in practice to be not adequate. Consider `unwrapBoxed : BoxedAdd → Int → Int → Int` from the example above⁵²: `unwrapBoxed` has an arity of 3, however, it should only have 1, otherwise if it were passed the additional two parameters, it would have no way to deal with it (as the body does not reference them.). In this case, only after η -expansion can it be allowed to have an arity of 3. Syntactic arity is much used in uncurried application.
4. **destructArgsPrelude** is used to destruct uncurried parameters – a single products. It produces several bindings that prepend the function body that represent the original parameters, as shown above with a consecutive `car/cdrs`. We shall discuss this topic later in closure conversion.
5. **lowerFCore** A call to which can be thought as living in the bottom of a continuation call stack. `lowerFCore` is *homoiconic* in the sense that it represents a tail in the lowering process, as well as lowers an expression that is the tail in the object language that is *Lambda IR*. It is defined as

```

@[inline] partial def lowerFCore
  (e : FExpr) (p : Env) (ctors : Std.HashMap String Nat) : M σ LExpr :=
  lowerF e p ctors $ pure ○ .seq #[] ○ .ret

```

Which, does not take a continuation to transfer control.

Implementing uncurrying is a joint effort on both function definition and application. We shall define the parameter layout: for a function of arity n , parameter $a_1, \dots, a_n$, we construct a tuple pl (payload) where

$$pl = \begin{cases} ((), ()) & n = 1, a_1 = () \\ (a_1, ()) & n = 1 \\ (a_1, \dots, a_n) & n \geq 2 \end{cases}$$

The author must admit that this is not exactly the best design in hindsight as it is List-like, but isn't exactly a list, especially for $n \geq 2$. Without typing information, it is harder to identify the shapes, a difficulty encountered later in codegen. Also, we should have encoded arity information

⁵⁰<https://proofassistants.stackexchange.com/a/4424>

⁵¹<https://github.com/leanprover/lean4/issues/5908>

⁵²Although tedious, we shall remind the readers of arrow types nest to the right.

inside function heads before *Lambda IR*, instead of collecting on-demand with less information and robustness, cluttering the code.

Since functions are now uncurried, we shall make a distinction between full applications and unsaturated (partial) ones: full applications are divided into several cases whereas partial application generates wrapper lambdas on demand. Interested readers shall refer to `lowerFunApp` in `ftransform.lean` that deal with various different scenarios of function application on a case-by-case basis: unary, partial, broken type/syntactic arity mismatch (called “broken” in code, the addition of dictionary parameters).

Constructor applications are done in a similar (as in partial application) way, but it is, at its core, not a function and does not have a body, which is why it is more simple to deal with.

Pattern matching is compiled using a *decision tree* approach⁵³ from Maranget. Based on this approach, we define three types that encapsulate the state of decision tree compilation and the tree itself: a `Sel` (similar to a `Proj` that encodes projection information), a `RowState` which encodes a row vector `a` in pattern matrix, and `DTree`, the tree itself.

```

inductive Sel where
  | base (idx : Nat)
  | field (s : Sel) (idx : Nat)
deriving Repr, Inhabited

structure RowState where
  pats : Array Pattern
  rhs  : FExpr
  binds : Array (String × Sel) := #[]
deriving Repr, Inhabited

inductive DTree where
  | fail
  | leaf (row : RowState)
  | testConst (j : Nat) (branches : Array (TConst × DTree)) (default? : Option DTree)
  | testCtor (j : Nat) (branches : Array (String × Nat × DTree)) (default? : Option DTree)
  | splitProd (j : Nat) (next : DTree)
deriving Repr, Inhabited

```

In PL jargon, the target of a pattern matching compilation is always called *matching automata*, which is either *decision trees* or *backtracking automata*. The latter being the simplest one, similar to one’s thinking process: from the top-leftmost of a matrix, test every pattern, then move on to the next row, until it reaches bottom-rightmost, which, has the advantage of maintaining linear space complexity in terms of *code size*. But, as stated in its name, it may backtrack, being potentially inefficient at runtime.

In contrast, we implement a decision tree compiler that makes sure no duplicate matching of the same value against the respective pattern. An example is taken from Maranget’s work to illustrate this issue: Consider

```

let prog
  | _, false, true  ⇒ 1
  | false, true, _  ⇒ 2
  | _, _, false     ⇒ 3
  | _, _, true      ⇒ 4

```

⁵³Exhaustive/Redundancy checking can be seen as a much lightweight algorithm derived from this approach

which is compiled to (only the relative parts are shown here; adjusted for better readability; $\emptyset$ denotes a default branch)

```

case x of
  false → case y of
    true  → case z of true → RET 2 | false → RET 2 |  $\emptyset$  → RET 2
    false → case z of true → RET 1 | false → RET 3
     $\emptyset$   → case z of true → RET 4 | false → RET 3
   $\emptyset$    → case y of
    false → case z of true → RET 1 | false → RET 3
     $\emptyset$   → case z of true → RET 4 | false → RET 3

```

We can see that, at worst, only 3 comparisons need to be made, much efficient compared to a backtracking automata.⁵⁴ It is worth noting that a lighter version of the algorithm described in the paper above is used in practice because some information has already been obtained during checking: the Match node of TExpr has a slot specifically to store the exhaustiveness of the matrix, sparing recalculations here. The wrapper type Subarray is used to minimize intermediate allocation of arrays.

4.2 Closure conversion of the *Lambda* IR

Closure conversion refers to hoisting locally bound functions to the toplevel, transforming it into a globally bound function. The main challenge is to eliminate free variables captured by local lambda abstractions.

While some text suggest that closure conversion is different from lambda lifting as the former does not add free variables to parameter list and does not adjust call sites, we use this word to describe the implementation *Tigris* used that takes the middle ground between lambda lifting and closure conversion, hence, these two words can be used interchangeably though we largely stick to the latter.

Some might suggest, since we are targeting Common Lisp, which is already a functional language itself with lambda abstractions, why the hassle – The reason is mostly a virtuous one – We reinvent the wheels to demonstrate the efforts put into the project, as a training in getting acquainted with practical implementations of functional languages as well as serving as an example for educational purposes.

The middle ground refers to the approach of lifting every Lam as well as adding an environment which stores all captured variables, this environment itself, is then passed alongside the function pointer, which, as a whole, is called a function object. When applying function objects, this environment (referenced as Γ) is assembled with uncurried parameters (referenced as *payload*) which is then passed to the function pointer.⁵⁵ This is called a middle ground because, although all free variables are eliminated, a function object is also created, though, the function itself is lifted to global. It is only when referencing⁵⁶ such functions, function objects are used instead of a bare function pointer. Besides, Lams are eliminated during this process, therefore, in spirit and in the author's opinion, this is an implementation of lambda lifting.

Function calling convention, or encoding of function objects, is described informally as follows

1. *payload* refers to the single parameter taken by lifted functions, $payload = \langle arg, \Gamma \rangle$ where *arg* refers to the product constructed from parameters

⁵⁴although this matrix can be further simplified e.g. simply return 2 when $(x, y, z) = (F, T, \cdot)$, but such optimization has not be implemented.

⁵⁵e.g. This pointer, in the SBCL backend, is simply a symbol that reference functions.

⁵⁶i.e. calling/passing.

2. Γ , or closure environment refers to explicit values holding captured free variables. It is encoded as a constructed value $\mathbf{E}[\![fv_1, \dots, fv_n]\!]$. In practice, to easily maintain its uniqueness, a non-parsable U+1D404 MATHEMATICAL BOLD CAPITAL E is used instead.
 Γ s are treated as if they are variants, encoded with `mkConstr`.
3. *Closure*, or *function object*, is a 2-field constructed value $\mathbf{C}[\![ptr, \Gamma]\!]$ where *ptr* refers to a code pointer (function pointer), which is a string internally. In practice, to easily maintain its uniqueness, a non-parsable U+1D402 MATHEMATICAL BOLD CAPITAL C is used instead.
Closures are treated as if they are variants, encoded with `mkConstr`.
4. *Application* of a closure project *ptr*, Γ and pass (arg, Γ) to *ptr*.
5. *Mutual recursion*, or recursion in general from `let rec` groups are converted so that each function body projects (arg, Γ) from payload. Calls from within the body reference the code pointer directly with the same env.⁵⁷ In case of mutual recursion, all functions in a `let rec` group shall share the same Γ layout.

Note that, since *Tigris* does not distinguish bindings of functional value from regular value, canonically speaking, every function definition in the form of a `let`-binding is no difference from binding names to lambda abstractions, which means, all functions i.e. all Lams are converted. In this sense, what would be called a “global” function collects free variables from the toplevel i.e. global definitions (if it is referenced). Consider

```
let x = 1;; let prog y = x + y + 1;; let main = prog 2
```

Though `x` and `prog` are both toplevel bindings, since the latter references the former, it is captured by Γ and projected inside `prog`’s body nonetheless, as shown in CC’d IR:

```
fn1000 (payload) {
  let  $\alpha$  = payload[0]
  let  $\Gamma$  = payload[1]
  let c0 =  $\Gamma$ [0]
  let _pL#y =  $\alpha$ [0]
  let p2 = ADD(c0, _pL#y)
  let c3 = 1
  let p4 = ADD(p2, c3)
  RET p4
}

main (payload) {
  let c0 = 1
  let  $\Gamma$  =  $\mathbf{E}[\![c0]\!]$ 
  let lam5 =  $\mathbf{C}[\![fn1000, \Gamma]\!]$ 
  let c6 = 2
  let u7 = ()
  let pair8 =  $\langle c6, u7 \rangle$ 
  _code1001 := lam5[0];
   $\Gamma$ c1002 := lam5[1];
  pc1003 :=  $\langle pair8, \Gamma c1002 \rangle$ ;
  _code1001(pc1003)T
}
```

⁵⁷This ensures efficient recursive calls as if they are native functions in the object language.

Remark (Performance concerns). Some might suggest that, for “globally defined functions” (in the traditional sense) that do not contain free variables (i.e. $\Gamma = \emptyset$), it is rather inefficient for them to be packaged inside a closure, then unwrapping them again for no reasons, instead, why not reference them directly⁵⁸ – The author wholeheartedly agree as *Tigris did*⁵⁹ have this optimization but it was considered experimental and issues did arise after enabling it, which is why it is temporarily removed, not that it matters a lot since most function calls are recursive calls and calls from within or from functions within a mutual group are already handled in this way.

Remark (Peephole: fuse immediate tailcall). A simple pattern matching on IR is used to detect: if an expression has the pattern $f^T(a)$ where f is the closure just bound for codeptr fid and env Γ , we rewrite this tail to $fid^T(\langle a, \Gamma \rangle)$; Old closure is handled and removed by DCE pass in the optimization module (which shall be discussed below).

```
def fuseImmediateTailCall (envName x fid : Name) : LExpr -> LExpr
| .seq binds (.app f a) =>
  if f == x then
    let pl := "p"
    .seq (binds.push (.let1 pl (.mkPair a envName))) (.app fid pl)
  else .seq binds (.app f a)
| .letVal y v b => .letVal y v (fuseImmediateTailCall envName x fid b)
| .letRhs y r b => .letRhs y r (fuseImmediateTailCall envName x fid b)
| .letRec fs b => .letRec fs (fuseImmediateTailCall envName x fid b)
| e => e
```

4.3 Optimizations of the *Lambda* IR

Optimizations are mostly done surrounding closure conversion: before CC and re-runs after to handle noises produced during CC. Currently, there are 3 passes: constant folding/propagation, copy propagation/dead code elimination (DCE) and tailcall-ify.

Constant folding & propagation refers to propagation of values that can be computed at compile time. Currently, the compiler handles $(+, -, *, \div)$ on integers and equality is handled for all primitive types: integer, bool, string. Although typically these operations come with laws such as associativity, commutativity, and distributivity, it is *not* respected by the optimizers i.e. $10+10+x+10$ will be simplified to $20+x+10$, but not $30+x$.

It would not be as interesting if the optimizer can’t interpret very basic control flow clauses such as conditional and pattern matching. e.g. `let x = true in if x then 1 else 2` is simplified to 1 and a more complex one:

```
type X = {x : Int, y : Int}
let s = {x = 1, y = 2}
let main =
  let {x, y} = s in x + y
```

Which gets simplified to 3. Constant folding is mostly useful when paired with typeclasses: consider

```
class Eq a = {eq : a -> a -> Bool}
type Option a = None | Some a
infixl 50 "==" => eq
```

⁵⁸by bare pointers i.e. names

⁵⁹See older commits in the repository.

```

let eqOpt = fun
  | Some x, Some y => x = y
  | None, None     => true
  | _, _           => false
instance forall a [Eq a], Eq (Option a) = {eq = eqOpt}

let main x y = Some x = Some y

```

if a method is registered by an accessible⁶⁰ variable, constant folding will be able to eliminate the dictionary projection of said method, replacing it with that variable, as both the instance and the method implementation `eqOpt` is visible to the optimizer. In this case, `(=)`, or `eq`, is replaced by `eqOpt` in `main`, making dictionary passing truly zero-abstraction cost.⁶¹

Copy propagation & dead code elimination is used in conjunction with other passes such as constant folding and closure conversion, as noted above. Copy propagation uses the transitivity of `(=)` in let-bindings, e.g. `let x = 1 in let y1 = x .. yn = yn-1 and z = 2 in yn + z` simplifies to 3. The optimizer knows that y_n alias y_{n-1} and *chases* this alias based on the law above, together with constant folding, yields 3.

Meanwhile, dead code elimination is as simple as it sounds: it eliminates unused bindings/expressions. Such as any unused global binding in `main` is removed, provided that there is not a transitive dependency. Although, it is worth noting that since *Tigris* is purely functional, DCE was used to be radical, removing any unused let-binding. This behavior is now changed if the LHS is a wildcard pattern as though *Tigris* is without `Seq`,⁶² let expression can be abused as in `let _ = act in ...` to run `act` with actual side effects such as IO, introduced by (the abuse of) the FFI system (described later).

Tailcall optimization is crucial for functional programming languages and is also guaranteed by many. Although readers are usually familiar with the topic if they are experience using such languages, we shall spare no trouble in explaining its importance.

Functional languages are differentiated from others by their extensive usage of functions (subroutines) in programs, which, when enter, pushes a new frame that serves as the context⁶³ of said call onto the call stack. The stack is unwinded as the control *returns* from the subroutine. When the call stack runs out of space, a *stack overflow* exception is thrown and the program is terminated. However, it turns out that calling a function in the *middle* of some code is different from calling it at the *end* since the latter can be seen as a transfer of control⁶⁴ that can be implemented by the machine instruction `j` instead of `call` (assuming RISC-V ISA), which, allows the stack to unwind once control is transferred, freeing up stack spaces.

Obviously, stack spaces are usually consumed by recursive calls. A recursive function is said to be *tail-recursive* if it ends with the call to itself.⁶⁵ *Tailcall optimization* refers to making sure functions like this (including non-recursive) are guaranteed to have stack space complexity $\mathcal{O}(1)$ rather than $\mathcal{O}(n)$. See examples: [Figure 3](#).

⁶⁰that is, it must be accessible at callsite, not shadowed by local binds.

⁶¹though it is not in this case. Since the `Option` instance for `Eq` is a function.

⁶²in source language, that is.

⁶³including callsites (return points) and stack-allocated values.

⁶⁴which, of course includes control structures such as pattern matching, conditional and non-local exit i.e. a transfer of control (and sometimes values) to an exit point for reasons other than a normal return, such as those produced by `goto`, `throw` and early returns.

⁶⁵in this case, frames are pushed and popped *consecutively*, between recursive calls.

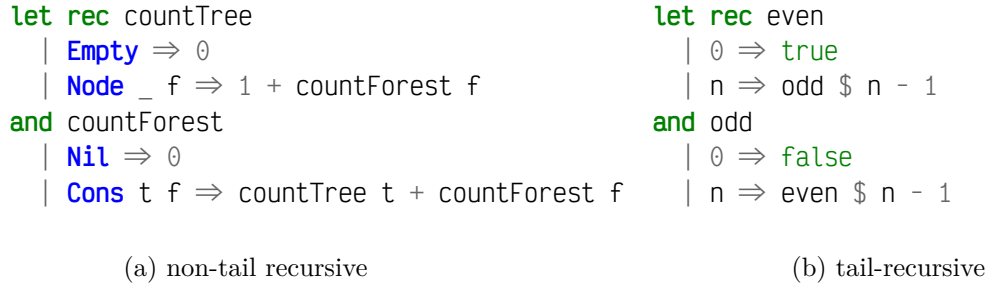


Figure 3: Non-tailcall vs. tailcall

Tailcall can be transformed, either by hand, or by mechanical CPS conversions (discussed later). Intuitively, tail recursive functions have similar though process and semantics to *iterations*.⁶⁶

Some might ask, from the description above, it seems that tailcall optimization is a rather low-level optimization. What does it mean when it is done at IR level, specifically, the *Lambda IR*?

The answer is that we *mark* tail call positions (encoded as `app`, from `Tail`) and pass down this information to the backend whom is responsible for making sure it stays in tail position in the object code that would otherwise be difficult to do so without the marks. A tailcall is represented, as previously mentioned, as $f(arg)^T$ where f is a function name or code pointer depending on the stage (before CC or after).

Optimizations are summed up by this method definition:

```

def optimizeLam (e : LExpr) : LExpr :=
  let e1 := constantFold e
  let e2 := copyPropDCE e1
  let e3 := tailcallify e2
  let e4 := copyPropDCE e3
  e4

```

We sum up this section by talking about the *ubiquitous* usage of CPS style in the lowering implementation, technical details are omitted as we shall only discuss the intuition here. The continuation function k , by homoiconicity, represent a *pluggable* expression (of the rest program) in the object language:

$$\begin{array}{ccc}
 k : & \text{Name} & \xrightarrow{\quad} \mu \sigma \text{ LExpr} & \text{(metalanguage; type only)} \\
 & \downarrow \text{plugging} & \downarrow \text{lowering} & \\
 \text{let} & [\bullet] & = \dots \text{ in } e & \text{(object language } \textit{Lambda IR})
 \end{array}$$

where the hole ($\bullet$) may or may not (but should, if not useless) be bound in e . Pluggable refers to the fact that applying such continuation to⁶⁷ some name (e.g. m) yields the rest of program with m bounded in e . Internally, it just runs through whatever monadic actions inside μ , in the explicit CPS stack, that, is maintained by a large closure whose body is the composition of continuation functions of the lowering actions, where, μ is defined by⁶⁸

```
abbrev M σ := StateRefT (Nat × AMap) (ST σ)
```

with `AMap` being the syntactic arity map from `Name` to `Nat`. In practice, this trick is used to *propagate* variable names generated by the metalanguage to the object language.

⁶⁶which, itself, is $\mathcal{O}(1)$ in stack space.

⁶⁷“applying to” can also be substituted for “plugging up”.

⁶⁸another use of the `ST` monad, reason same as before.

Incremental APIs and the *Tigrisi* IR interpreter is a experimental extension. While the *Tigrisl* compiler compiles programs in one go, it is rather inefficient for REPLs as they have different use logic: users of a REPL session usually enter a program *piece by piece*, while expecting immediate result after entering them. A naive REPL might track everything they entered since the start of REPL then *re-compiles* it per user entrance, which, for multi-stages language implementations with several passes is horribly inefficient. Thus, the incremental APIs are born, which lives in the Incremental namespace.

When implementing the incremental APIs, we not only have to track every state ⁶⁹, but also the use of it inside `{ftransform, lift, opt}.lean` (is it intertwined, etc):

```
structure LoweringState where
  gensym : Nat
  env     : Env
  ctors   : Std.HashMap String Nat
  arity   : Std.HashMap String Nat
  deriving Inhabited
```

Interested users shall refer to them in the bottoms of `ftransform.lean` and `lift.lean`. This is considered highly experimental and unstable since it is not thoroughly tested.

the *Tigrisi* IR interpreter targets the *Lambda* IR and can run independently regardless of with incremental APIs or not. Though in the other case it runs much slower due to worse compilation performance. The interpreter itself is stable, but due to its partial dependency on the incremental APIs, we still consider it experimental.

Interpreters, just like virtual machines, have states, such as the OCaml VM that runs the bytecode produced by `ocamlc`. Same is applied to *Tigris*. Largely, these states are introduced by bindings, whether local or global, or functional. In this case, the state is a *mapping*, or *table* (e.g. a Hashmap or Treemap). An important design aspect to consider is to make the state as lightweight as possible, since allocating them, projecting/modifying them consumes time and space. In our implementation, we define immediate values and environments as:

```
inductive Val where
  | unit
  | int (i : Int)
  | bool (b : Bool)
  | str (s : String)
  | pair (p q : Val)
  | ctor (tag : Name) (fields : Array Val)
  | code (pointer : Name)
  deriving Repr, BEq, Inhabited
```

```
abbrev Env      := Std.HashMap Name Val      -- local binding table
abbrev FunTab   := Std.HashMap Name LFun     -- Function table
abbrev GlobalEnv := Std.HashMap Name Val     -- global binding table
```

On encountering a `let*`⁷⁰, relevant bindings are inserted to one of the three tables defined above.

Tigrisi runs in `EIO`⁷¹ and is *multi-threaded*: evaluation runs in a separate thread. This way it can respond to the keystroke `<C-C>` from user, on encountering which the REPL thread is terminated.

⁶⁹and temporary states that is dropped after the compilation of a program, but should be saved in this case.

⁷⁰* is wildcard, matching empty string, ι or ω , as in the pretty-printed syntax of *Lambda* IR. It has nothing to do with sequential application from LISP or OCaml.

⁷¹that is, `EIO` with exception handling.

Termination of threads is *cooperative*: A joint effort of the interpreter, through `checkInterrupt` up front at crucial branch (`eval*` functions):

```

1  /-! leval.lean -/
2  private def checkInterrupt : EIO TypingError Unit :=
3      IO.checkCanceled >>= fun
4          | true  => throw .Interrupted
5          | false => return ()
6
7  /-! ffidecl.lean -/
8  @[extern "lean_sigint_pipe"]
9  opaque installSigintPipe : IO Int32
10 @[extern "lean_read_fd_byte"]
11 opaque readFdByte : @&Int32 → IO Int32
12
13 /-! Tigrisi.lean -/
14 -- (.. within main ..)
15 let stdin ← getStdin
16 let replST ← mkRef LApp.initREPL
17 let EVS : IO.Ref EvalState ← mkRef none
18
19 let fd ← installSigintPipe
20
21 if fd ≥ 0 then
22     discard $ asTask (prio := .dedicated) do
23         repeat do
24             let r ← readFdByte fd
25             if r ≤ 0 then pure ()
26             else
27                 if let some t ← EVS.get then
28                     IO.cancel t
29                 else pure ()
30 -- (..)

```

with the help of CLI frontend (above) and Lean’s C FFI interface (we omit the win32 implementation):⁷²

```

1  typedef lean_obj_res OBJRES;
2  /* (.. win32 impl ..) */
3  static int32_t sig_pipe[2] = {-1, -1};
4
5  static void sigint_handler() {
6      // write is async-signal-safe
7      char b = 1;
8      if (sig_pipe[1] >= 0) {
9          (void)!write(sig_pipe[1], &b, 1);
10     }
11 }
12
13 LEAN_EXPORT OBJRES lean_sigint_pipe() {

```

⁷²this approach is adopted from the deprecated REPL *tigris* (which is a subset of the current *Tigris*).

```

14     if (sig_pipe[0] != -1)
15         return lean_io_result_mk_ok(lean_box(sig_pipe[0])); // already installed
16     if (pipe(sig_pipe) != 0)
17         return lean_mk_io_user_error(
18             lean_mk_string("sigint pipe installation failed"));
19     struct sigaction sa;
20     memset(&sa, 0, sizeof(sa));
21     sa.sa_handler = sigint_handler;
22     sigemptyset(&sa.sa_mask);
23     sa.sa_flags = SA_RESTART; // restart interrupted syscalls
24     if (sigaction(SIGINT, &sa, NULL) != 0) {
25         close(sig_pipe[0]);
26         close(sig_pipe[1]);
27         sig_pipe[0] = sig_pipe[1] = -1;
28         return lean_mk_io_user_error(lean_mk_string("call to sigaction() failed"));
29     }
30     return lean_io_result_mk_ok(lean_box(sig_pipe[0]));
31 }
32
33 LEAN_EXPORT lean_obj_res lean_read_fd_byte(int32_t fd) {
34     char b;
35     for (;;) {
36         ssize_t r = read(fd, &b, 1);
37         if (r == 1)
38             return lean_io_result_mk_ok(lean_box(1));
39         if (r == 0)
40             return lean_io_result_mk_ok(lean_box(0)); // EOF
41         if (errno == EINTR)
42             continue; // retry
43         return lean_mk_io_user_error(lean_mk_string("read() failed"));
44     }
45 }

```

The CLI frontend is responsible for launching evaluation threads EVS as well as a *sentinel thread* that keeps reading the sigint pipe, on detecting cancel signals from the user, signals EVS to exit, which is then responded by `checkInterrupted` within the thread, who exits *gracefully* i.e. through normal returns. EVS is a threadlocal mutable state `IO.Ref EvalState` where `EvalState` is defined as

```

abbrev EvalState := Option
    $ Task
    $ Except Error
    $ LApp.REPLState × LInterpreter.Val × Scheme

```

where, `Task` is a Lean primitive representing asynchronous computation, similar to *coroutines*, such as Scala's `Future`, Javascript's `Promise` and Rust's `JoinHandle`. The POSIX sigint pipe can only be read if it is installed. Implementation of this pipe, its handler and reader is in C, through the FFI to be called in Lean, as shown above.

Now, if some computation diverges, e.g., `let rec loop x = loop x in loop ()`, users can press `<C-C>` to interrupt a thread: [Figure 4](#). Though, users should note that there is a caveat: an tigris REPL launched through Lake⁷³ has broken sigint pipe where `<C-C>` just terminates the program,

⁷³Lean's build system

```

notchip in pop-os122 in tigris on / cwc [!+] via λ 9.6.7 via v5.3.0 (default)
[7s] $ .lake/build/bin/tigris
A basic language using Hindley-Milner type system
with a naive (term-rewriting) interpreted implementation.
For language specifications see source.
USAGE
  • Type #help;; to check available commands.
  • Exit with <C-d> or <C-z-RET> (Windows).
  • Interrupt with <C-c> (doesn't work if
    running through 'lake exe')
> let rec id x = id x in id 20;;
^C[ERROR] Interrupted.
>

```

Figure 4: A user interrupting a computation

instead, users should launch the program directly, which, should be the default behavior if not building from source.

The *Tigris* interpreter can be invoked with `interpretL` (w/o incr. APIs) or `interpretI`, of which the former is stable and is used to quickly test the lowering stage to see if there is any issue.

4.4 CPS transformation

CPS transformation refers to the transformation of *Lambda* IR into its CPS counterparts where

- every function takes an explicit continuation parameter k ,
- function calls pass results to continuations rather than returning,
- continuations are *not* first-class: they are either locally-bound labels or the special parameter k ,
- ANF is maintained.

The use of CPS, as explained in the section about tailcall, is to transform non-tailrec functions. This is rather specific as in general, mechanized CPS is used to encode complex control flow, similar to a longjmp and CPS transformation is to make control flow explicit, at the expense of increased heap usage, as continuations are usually lambda abstraction⁷⁴ them selves, being passed around together with function parameters. The intuition is that, CPS trades heap space for stack space, by allocating and *compositing* continuations, which, can be thought as

an explicit call stack (that replaces the actual call stack) which is unwinded when applied.

The CPS IR is similar to *Lambda* IR, defined as (CName is Name)

```

1  inductive CRhs where
2    | prim      (op : PrimOp) (args : Array CName)
3    | proj      (src : CName) (idx : Nat)
4    | mkPair    (a b : CName)
5    | mkConstr  (tag : Name) (fields : Array CName)
6    | isConstr  (src : CName) (tag : Name) (arity : Nat)
7    | const     (k : Const)
8    | alias     (y : CName) -- x = y
9  deriving Repr, Inhabited, BEq
10
11  mutual

```

⁷⁴which usually is allocated in heap. Though most compilers may force stack allocation if there is an optimization window, that defeats the purpose of using CPS to save stack space if it is too naive.


```

12  inductive CTail where
13  | appFun      (f : CName) (payload : CName) (k : CName)
14  | appKont     (k : CName) (value : CName)
15  | ite         (condVar : CName) (tBranch eBranch : CExpr)
16  | switchConst (scrut : CName)
17                (cases : Array (Const × CExpr))
18                (default? : Option CExpr)
19  | switchCtor  (scrut : CName)
20                (cases : Array (Name × Nat × CExpr))
21                (default? : Option CExpr)
22  | halt        (value : CName)
23  | matchFail   (pat : Array Name)
24  deriving Repr, Inhabited, BEq
25
26  /-- CPS expression in ANF: pure let1 / letKont / local fun groups, ending with a tail. -/
27  inductive CExpr where
28  | let1      (x : CName) (rhs : CRhs) (body : CExpr)
29  | letKont   (kid : CName) (param : CName) (kBody : CExpr) (body : CExpr)
30  | letRec    (funs : Array CFun) (body : CExpr)
31  | tail      (t : CTail)
32  deriving Repr, Inhabited, BEq
33
34  structure CFun where
35  fid      : CName
36  payloadParam : CName
37  kontParam  : CName
38  body      : CExpr
39  deriving Repr, Inhabited, BEq
40  end

```

The CPS transformation is the last step of the IR lowering process, which, is further lowered by the SBCL backend to Common Lisp. Semantics of the actual program should be similar to that of CPS, provided if there is not any backend inconsistency⁷⁵

First, we present the formal grammar of *Lambda* IR, and its CPS target:

$$\begin{aligned}
v \in \text{Val} &::= x \mid k \mid \text{ctor}(t, \bar{x}) \mid \lambda p. e \\
r \in \text{Rhs} &::= \text{prim}(op, \bar{x}) \mid \text{proj}(x, i) \mid \text{mkPair}(x, y) \\
&\quad \mid \text{mkCtor}(t, \bar{x}) \mid \text{isCtor}(x, t, n) \mid f(a) \\
s \in \text{Stmt} &::= \text{let}_1 x = r \\
\tau \in \text{Tail} &::= \text{ret } x \mid f(a)^\top \\
&\quad \mid \text{if } c \text{ then } e_1 \text{ else } e_2 \\
&\quad \mid \text{switch}^{\text{Const}}(x, \overline{k_i \rightarrow e_i}, e?) \\
&\quad \mid \text{switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow e_i}, e?) \\
&\quad \mid \text{matchFail}
\end{aligned}$$

⁷⁵which, unfortunately, remains an assumption because *Tigris* is not formally verified. Note that, the M monad is continued to use for this transformation.

$$\begin{aligned}
e \in \text{LEpr} &::= \bar{s}; \tau \\
&| \text{let}_\ell x = v \text{ in } e \\
&| \text{let}_1 x = r \text{ in } e \\
&| \text{let rec } \overline{f_i(p_i) = e_i} \text{ in } e \\
r_c \in \text{CRhs} &::= \text{prim}(op, \bar{x}) \mid \text{proj}(x, i) \mid \text{mkPair}(x, y) \\
&| \text{mkCtor}(t, \bar{x}) \mid \text{isCtor}(x, t, n) \mid k \mid x \\
\tau_c \in \text{CTail} &::= f(p, k) \mid k(v) \mid \text{halt}(v) \\
&| \text{ite}(c, e_1, e_2) \\
&| \text{switch}^{\text{Const}}(x, \overline{k_i \rightarrow e_i}, e?) \\
&| \text{switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow e_i}, e?) \\
&| \text{matchFail} \\
e_c \in \text{CEpr} &::= \text{let}_1 x = r_c \text{ in } e_c \\
&| \text{letK } \kappa(p) = e_c \text{ in } e_c \\
&| \text{let rec } \overline{f_i(p_i, k_i) = e_i} \text{ in } e_c \\
&| \text{tail } \tau_c
\end{aligned}$$

We define the *domain of semantics* (i.e. datatypes) as:⁷⁶

$$\begin{aligned}
\text{Val} &= () + \mathbb{Z} + \mathbb{B} + \text{String} + (\text{Val} \times \text{Val}) + (\text{Name} \times \text{Val}^*) + \text{Name} \\
\rho &= \text{Name} \rightarrow \text{Val} \\
\Phi &= \text{Name} \rightarrow (\text{Name} \times \text{Name} \times \text{CEpr}) \\
\Phi_\kappa &= \text{Name} \rightarrow (\text{Name} \times \text{CEpr}) \\
\text{Ans} &= \text{Val} + \text{Error}
\end{aligned}$$

The transformation $\mathcal{C} : \text{LEpr} \rightarrow \text{CEpr}$ is defined with respect to a *current continuation* k , which is either a continuation variable or should be the identity continuation id_{Ans} ⁷⁷ for top-level expressions.

Below, we present the denotational semantics for this transformation. Note that $C_{[\bullet]}^k$ represents a transformation with bound continuation k , with a hole denoting an element of the input domain $[\bullet]$, which is one of v, r, s, τ, e ; And $C_s^k[\bullet](e_c)$ threads e_c . We simplify let^* and switch^* to just one binding/one discriminant.

$$\begin{aligned}
\mathcal{C}_v[\![x]\!] &= x & (\text{Values}) \\
\mathcal{C}_v[\![k]\!] &= \text{const}(k) \\
\mathcal{C}_v[\![\text{ctor}(t, \bar{x})]\!] &= \text{mkCtor}(t, \bar{x}) \\
\mathcal{C}_r[\![\text{prim}(op, \bar{x})]\!] &= \text{prim}(op, \bar{x}) & (\text{Rhs}) \\
\mathcal{C}_r[\![\text{proj}(x, i)]\!] &= \text{proj}(x, i) \\
\mathcal{C}_r[\![\text{mkPair}(x, y)]\!] &= \text{mkPair}(x, y) \\
\mathcal{C}_r[\![\text{mkCtor}(t, \bar{x})]\!] &= \text{mkCtor}(t, \bar{x}) \\
\mathcal{C}_r[\![\text{isCtor}(x, t, n)]\!] &= \text{isCtor}(x, t, n)
\end{aligned}$$

⁷⁶think of $(+)$ and $(\times)$ as coproduct and product, respectively.

⁷⁷The standard *driver* in SBCL backend is this.

$$\begin{aligned}
\mathcal{C}_r[f(a)] &= (\text{see call transformation below}) \\
\mathcal{C}_\tau^k[\text{ret } x] &= \text{tail } k(x) & (\text{Tails}) \\
\mathcal{C}_\tau^k[f(a)^\top] &= \text{tail } f(a, k) \\
\mathcal{C}_\tau^k[\text{if } c \text{ then } e_1 \text{ else } e_2] &= \text{tail ite}(c, \mathcal{C}_\tau^k[e_1], \mathcal{C}_\tau^k[e_2]) \\
\mathcal{C}_\tau^k[\text{switch}^{\text{Const}}(x, \overline{k_i \rightarrow e_i, e?})] &= \text{tail switch}^{\text{Const}}(x, \overline{k_i \rightarrow \mathcal{C}_\tau^k[e_i]}, \mathcal{C}_\tau^k[e?]) \\
\mathcal{C}_\tau^k[\text{switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow e_i, e?})] &= \text{tail switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow \mathcal{C}_\tau^k[e_i]}, \mathcal{C}_\tau^k[e?]) \\
\mathcal{C}_\tau^k[\text{matchFail}] &= \text{tail matchFail} \\
\mathcal{C}_s^k[\bar{s}; \tau] &= \mathcal{C}_s^k[\bar{s}](\mathcal{C}_\tau^k[\tau]) & (\text{Stmt}) \\
\mathcal{C}_s^k[\epsilon](e_c) &= e_c \\
\mathcal{C}_s^k[(\text{let}_1 x = r); \bar{s}](e_c) &= \begin{cases} \text{letK } \kappa(x) = \mathcal{C}_s^k[\bar{s}](e_c) \text{ in tail } f(a, \kappa) & r = f(a) \\ \text{let}_1 x = \mathcal{C}_r[r] \text{ in } \mathcal{C}_s^k[\bar{s}](e_c) & \text{otherwise} \end{cases} \\
\mathcal{C}^k[\text{let}_t x = v \text{ in } e] &= \text{let}_1 x = \mathcal{C}_v[v] \text{ in } \mathcal{C}^k[e] & (\text{letVal}) \\
\mathcal{C}^k[\text{let}_1 x = f(a) \text{ in } e] &= \text{letK } \kappa(x) = \mathcal{C}^k[e] \text{ in tail } f(a, \kappa) & (\text{letRhs}) \\
\mathcal{C}^k[\text{let}_1 x = r \text{ in } e] &= \text{let}_1 x = \mathcal{C}_r[r] \text{ in } \mathcal{C}^k[e] \quad (r \neq f(a)) \\
\mathcal{C}^k[\text{let rec } f(p) = e_f \text{ in } e] &= \text{let rec } f(p, k_f) = \mathcal{C}^{k_f}[e_f] \text{ in } \mathcal{C}^k[e] & (\text{letRhs})
\end{aligned}$$

Finally, we present the denotational semantics of CPS IR *itself*. Just as above, the semantics is given by

$$\begin{aligned}
\mathcal{E} : (\text{CExpr} \oplus \text{CTail}) &\rightarrow \rho \rightarrow \Phi \rightarrow \Phi_\kappa \rightarrow (\text{Val} \rightarrow \text{Ans}) \rightarrow \text{Ans} \\
\mathcal{R} : \text{CExpr} &\rightarrow \rho \rightarrow \text{Val}
\end{aligned}$$

where the fifth parameter i.e. of type $\text{Val} \rightarrow \text{Ans}$ is the *outermost continuation*, or the *meta-continuation*, denoted μ , which represents the toplevel continuation that is id_{Ans} or halt . We write $\rho[n \mapsto v]$ to denote the insertion⁷⁸ of mapping $n \mapsto v$ to ρ ; Readers should acknowledge that ρ, n, v are metavariables. Note that $\mathcal{E}$ takes both tails and expressions.

$$\begin{aligned}
{}^{79}\mathcal{R}[\text{prim}(op, \bar{x})]\rho &= \delta(op, \rho(\bar{x})) & (\text{CRhs}) \\
\mathcal{R}[\text{proj}(x, i)]\rho &= \pi_i(\rho(x)) \\
\mathcal{R}[\text{mkPair}(x, y)]\rho &= (\rho(x), \rho(y)) \\
\mathcal{R}[\text{mkCtor}(t, \bar{x})]\rho &= (t, \rho(\bar{x})) \\
\mathcal{R}[\text{isCtor}(x, t, n)]\rho &= \begin{cases} \text{true} & \rho(x) = (t, \bar{v}) \wedge |\bar{v}| = n \\ \text{false} & \text{otherwise} \end{cases} \\
\mathcal{R}[\text{const}(k)]\rho &= k \\
\mathcal{R}[\text{alias}(y)]\rho &= \rho(y) \\
\mathcal{E}[f(p, k)]\rho\Phi\Phi_\kappa\mu &= \begin{cases} \mathcal{E}[e_f]\rho'\Phi'\Phi'_\kappa\mu & \Phi(f) = (p_f, k_f, e_f) \\ \text{Error} & \text{otherwise} \end{cases} & (\text{funapp})
\end{aligned}$$

⁷⁸insertion follows the standard behavior: if the key m exists, its value is replaced by v .

$$\begin{aligned}
& \text{where } \rho' = \{p_f \mapsto \rho(p), k_f \mapsto \rho(k)\} \\
& \Phi' = \Phi \\
& \Phi'_\kappa = \emptyset \quad (\text{local continuations are scoped}) \\
& {}^{80}\mathcal{E}[\![\kappa(v)]\!]\rho\Phi\Phi_\kappa\mu = \begin{cases} \mathcal{E}[e_\kappa]\rho[p_\kappa \mapsto \rho(v)]\Phi\Phi_\kappa\mu & \Phi_\kappa(\kappa) = (p_\kappa, e_\kappa) \\ \mu(\rho(v)) & \kappa = k_{\text{param}} \\ \text{Error} & \text{otherwise} \end{cases} \quad (\text{kontapp}) \\
& \mathcal{E}[\![\text{halt}(v)]\!]\rho\Phi\Phi_\kappa\mu = \rho(v) \quad (\text{halt}) \\
& \mathcal{E}[\![\text{ite}(c, e_1, e_2)]\!]\rho\Phi\Phi_\kappa\mu = \begin{cases} \mathcal{E}[e_1]\rho\Phi\Phi_\kappa\mu & \rho(c) = \text{true} \\ \mathcal{E}[e_2]\rho\Phi\Phi_\kappa\mu & \text{otherwise} \end{cases} \quad (\text{conditional}) \\
& \mathcal{E}[\![\text{switch}^{\text{Const}}(x, \overline{k_i \rightarrow e_i}, e?)]\!]\rho\Phi\Phi_\kappa\mu = \begin{cases} \mathcal{E}[e_j]\rho\Phi\Phi_\kappa\mu & \exists j. \rho(x) = k_j \\ \mathcal{E}[e?]\rho\Phi\Phi_\kappa\mu & \forall i. \rho(x) \neq k_i \wedge e? \text{ exists} \\ \text{Error} & \text{otherwise} \end{cases} \quad (\text{const switch}) \\
& \mathcal{E}[\![\text{switch}^{\text{ctor}}(x, \overline{(t_i, n_i) \rightarrow e_i}, e?)]\!]\rho\Phi\Phi_\kappa\mu = \begin{cases} \mathcal{E}[e_j]\rho\Phi\Phi_\kappa\mu & \rho(x) = (t_j, \bar{v}) \wedge |\bar{v}| = n_j \\ \mathcal{E}[e?]\rho\Phi\Phi_\kappa\mu & \text{no match; } e? \text{ exists} \\ \text{Error} & \text{otherwise} \end{cases} \quad (\text{ctor switch}) \\
& \mathcal{E}[\![\text{matchFail}]\!]\rho\Phi\Phi_\kappa\mu = \text{MATCH-FAILURE} \quad (\text{matchfail}) \\
& \mathcal{E}[\![\text{let}_1 x = r \text{ in } e]\!]\rho\Phi\Phi_\kappa\mu = \mathcal{E}[e]\rho[x \mapsto \mathcal{R}[r]\rho]\Phi\Phi_\kappa\mu \quad (\text{let1}) \\
& {}^{81}\mathcal{E}[\![\text{letK } \kappa(p) = e_\kappa \text{ in } e]\!]\rho\Phi\Phi_\kappa\mu = \mathcal{E}[e]\rho\Phi\Phi_\kappa[\kappa \mapsto (p, e_\kappa)]\mu \quad (\text{letKont}) \\
& \mathcal{E}[\![\text{let rec } f(p, k_f) = e_f \text{ in } e]\!]\rho\Phi\Phi_\kappa\mu = \mathcal{E}[e]\rho\Phi[f \mapsto (p, k_f, e_f)]\Phi_\kappa\mu \quad (\text{letRec}) \\
& \mathcal{E}[\![\text{tail } \tau]\!]\rho\Phi\Phi_\kappa\mu = \mathcal{E}[\![\tau]\!]\rho\Phi\Phi_\kappa\mu \quad (\text{tail})
\end{aligned}$$

Experienced readers should immediately notice some key properties based on the definition of this CPS IR, nevertheless, we list them as remarks below:

Remark (Non-first class continuations). refer to continuations appear only in specific syntactic positions, as listed below:

1. As the second argument to function calls: $f(p, k)$
2. As the target of continuation application: $\kappa(v)$
3. Bound by **letK**: $\text{letK } \kappa(p) = e \text{ in } \dots$

This is enforced by the definition (abstract syntax) of CPS IR, where there is no CRhs that captures a continuation as a value. Some users might be disappointed as such IR could not support advanced control primitives such as `call/cc` or `shift/reset` from delimited continuation. Although such primitives can be more efficiently treated by the compiler, this form of CPS really is not widely used and can be substituted for more boilerplate code. Even if one must use it, we suggest an *internalized* version `Cont` which is more local with good maintainability⁸² through the `Monad` interface, similar to the one from `Control.Monad.Cont` in `GHC`. Refer to testcase `examples/cont.tig` for an example.

⁷⁹where δ is the interpretation of primitive operations and π_i is projection.

⁸⁰The second case handles the distinguished continuation parameter: applying it invokes the toplevel continuation μ , effectively returning from the current function.

⁸¹Note that continuations are *not* values in the environment; they are stored in a separate continuation table Φ_κ , reflecting that they are not first-class.

⁸²since CPS itself can be convoluted and counterintuitive of which the incautious use is just as bad as `goto`.

Remark (Guaranteed tailcall). refers to the fact that all function calls in CPS are in tail position.

$\mathcal{E}[\![f(p, k)]\!] \dots$ directly invokes f without building stack frames

Non-tail calls in the source are transformed via **letK**:

$$\mathbf{let}_1 x = f(a) \mathbf{in} e \quad \rightsquigarrow \quad \mathbf{letK} \kappa(x) = C^k[\![e]\!] \mathbf{in} \mathbf{tail} f(a, \kappa)$$

where the continuation κ captures the rest of the computation e .

Remark (Closure representation after CPS). Previously, we discussed the encoding and calling convention of function objects, this, is largely unchanged after CPS:

$$C[\![ptr, \Gamma]\!]$$

where Γ is the captured environment. In CPS, function calls become:

$$\mathcal{E}[\![f(p, k)]\!] = \dots \text{ where } f \text{ is a code pointer}$$

The payload p is a pair (α, Γ) where α contains the user arguments.

Remark (CPS Module semantics). A CPS module consists of:

$$\begin{aligned} M &= (\bar{F}, F_{main}) \\ F &= (f, p, k, e) \end{aligned}$$

The semantics of a module is:

$$\mathcal{M}[\![\bar{F}, F_{main}]\!] = \mathcal{E}[\![e_{main}]\!]\rho_0\Phi_0\mathcal{O}\mathbf{id}_{Ans}$$

where:

$$\begin{aligned} \Phi_0 &= \{f_i \mapsto (p_i, k_i, e_i) \mid F_i \text{ of } (f_i, p_i, k_i, \cdot) \in \bar{F}\} \cup \{f_{main} \mapsto (p_m, k_m, e_m)\} \\ \rho_0 &= \{p_m \mapsto ((), \mathbf{E}[\![]]\!])\} \end{aligned}$$

The initial payload is $(((), ()), \mathbf{E}[\![]]\!])$ ⁸³. Though, this behavior can be changed in the backend by modifying the driver wrapper code to pass whatever value to `main` in Common Lisp.

4.5 Backend: Common Lisp (SBCL) & FFI

Before proceeding further, let us recap all the stages in the implementation of Tigris, through datatypes:

$$\begin{aligned} \text{Tigris source} &\xrightarrow{\textcircled{1}} \text{Expr} \xrightarrow{\textcircled{2}} \text{TExpr} \xrightarrow{\textcircled{3}} \text{FExpr} \xrightarrow{\textcircled{4}} \text{LExpr} \xrightarrow{\textcircled{5}} \text{CExpr} \xrightarrow{\textcircled{6}} \text{LISP source} \\ \text{where } \textcircled{1} &: \text{lexing, parsing} \quad \textcircled{2} : \text{typechecking} \quad \textcircled{3} : \text{instance resolution, SysF elaboration} \\ \textcircled{4} &: \text{Lambda IR lowering} \quad \textcircled{5} : \text{CPS conversion} \quad \textcircled{6} : \text{SBCL codegen} \end{aligned}$$

The SBCL backend simply translate CPS IR to Common Lisp source. Since SBCL is functional and dynamically typed, specialization or most type information is not needed, continuations are created as-is and *defunctionalization* is not attempted. The reason for choosing Common Lisp instead of other languages such as JS or Go is because the author considers himself a LISPer, who enjoys using macros.

Informally, we describe the translation as follows:

⁸³Recall, for function with parameter a of arity 1, we construct $(a, ())$ even if $a = ()$.

toplevel functions refer to all functions, of shape $fid(pl, k)$, as `(defun fid (pl k))`;

let-binding i.e. `let1` as the standard `(let ((n v)) ..)`;

kont-binding i.e. `letKont` $k \ v = ..$ is translated to `(labels ((k (v) ..)) body)`;

kont-app i.e. `APPLY` $k \ v$ as `(funcall (the function k) v)`

closure i.e. $C[[ptr, \Gamma]]$ as `#'ptr` if ptr is toplevel/label-bound, or otherwise, store as-is;

Γ i.e. $E[[\bar{f}]]$, for each field f ,

- store as-is if f is a local functional value,
- store `#'f` if f is toplevel/label-bound,
- store as-is, otherwise.

funcall i.e. $f(a, k)$ as `(funcall (the function f) ..)` where f is local, or `(funcall #'f ..)` where f is toplevel/label-bound

conditionals i.e. $ite(c, t, e)$ as `(if c t e)`

switch where

- $switch^{Const}(x, \overline{k_i \rightarrow e_i}, e_?)$ as `(cond (((equal x k_i) e_i) .. (t e_?)) ..)`
- $switch^{ctor}(x, \overline{(t_i, n_i) \rightarrow e_i}, e_?)$ `(cond (((eq x t_i) e_i) .. (t e_?)) ..)`

product i.e. $pr = (p, q)$ to `(cons p q)` and projects with the standard `(car pp)` or `(cdr pp)`

variant i.e. $ctag[[fld]]$ as `(cons tag (vector @fld))`⁸⁴ with projection x_i as `(svref (cdr x) i)`. E and C are encoded this way as well.

test-ctor i.e. $isCtor(x, tag,)$ as `(eq (car x) tag)`. Note that arity is not checked at this time. *Symbols* appear inside CPS IR are escaped with bars `||` to preserve the case, as in

```
def sym (s : Name) : String :=
  let esc := s.foldl (init := "|") fun acc ch =>
    match ch with
    | '|' => acc ++ "\\|"
    | '\\' => acc ++ "\\\\"
    | _ => acc.push ch
  esc.push '|'
```

The caveats, as described in [section 4.1](#) (uncurrying), because type information is erased, we collect *shapes* on demand to *alleviate* this problem, where Shapes can be thought as degenerate types

⁸⁴as in `'simple-vector`, the fastest stock array variant. Note that `@x` denotes the splice of x .

```

inductive Shape where
  | unknown
  | pair
  | ctor (tag : Name) (arity : Nat)
  | fn -- function object inside value cell
deriving Inhabited, BEq, Repr
abbrev ShapeEnv := Std.HashMap Name Shape

```

where ShapeEnv associates a name with a shape. The initial shape of a name is always unknown, but will be *refined* as the code is being compiled. However we use the word *alleviate* because this method is not robust as it is local to functions and does not deal with the root cause: design flaws. Major refactor of adding type information to Lambda IR, threading through CPS transformation and codegen must be made to address the issue.

The runtime system refers to the primitive operator implementations, driver code and the definitions of low-level exceptions, e.g. the string equality %string=, the integer addition %int+ and the MATCH-FAILURE condition +NOMATCH+⁸⁵ is implemented as

```

1  (declaim (inline %string=))
2  (defun %string= (s1 s2)
3    (declare (type simple-string s1 s2))
4    (the boolean (sb-kernel:%sp-string= s1 s2 0 nil 0 nil)))
5
6  (declaim (ftype (function (integer integer) integer) %int+) (inline %int+))
7  (defun %int+ (a b)
8    (declare (optimize (speed 3) (debug 0) (safety 0)))
9    (sb-kernel:two-arg-+ a b))
10
11 (define-condition match-failure (error)
12   ((discriminant :initarg :discr :reader discriminant :type string))
13   (:report
14    (lambda (condition stream)
15      (format stream "No branch can be matched against ~A" (discriminant condition))))))
16 (defconstant +NOMATCH+ 'match-failure)

```

Interested readers should refer to runtime.lisp.

Common Lisp have all kinds of equality relations. To minimize overhead on dynamically typed languages, we largely follow Table 7. EQ is the most efficient as it compares memory address (i.e. physical equality rather than structural equality), which, is used most, in our case, such as comparing constructors as they are always unique, which has the same memory address. Compared to the usual encoding of constructors as integers, we achieve same performance here with better readability since the name of the constructor is the symbol itself.

Remark (Inlining runtime). The content of runtime.lisp is *inlined* in the compiler, through a trick used in the build script (also in Lean.) which detects whether there is a change in the file, and writes to runtime.lean if so:

```

1  input_file runtime.lisp where
2  path := __dir__ / "runtime.lisp"

```

⁸⁵Common Lisp have *conditions*, which, is arguably more expressive and general than other languages' exception system as it encodes all kinds of control flow, not just exceptions. In fact, it can be used to implement an internalized continuation facility.

To compare against..	Use..
Objects/Structs	EQ
Symbols	EQ
NIL	EQ/NULL
T	EQ
Precise numbers	EQL
Floats	=
Char	EQL/CHAR=
List/Cons/Seq	EQUAL
String	EQUAL/EQUALP/STRING= (symbols as well)
Tree	TREE-EQUAL

Table 7: LISP equalities

```

3   text := true
4   target runtime.lean : Unit := do -- NOTE: see also `meta` (modifier)
5     let runtimep ← runtime.lisp.fetch
6     let rstrBegin := "r###\n".toUTF8
7     let rstrEnd   := "\n###".toUTF8
8     runtimep.mapM fun path =>
9       IO.FS.withFile path .read fun rt =>
10        IO.FS.withFile (__dir__ / "runtime.lean") .write fun h => do
11          let runtime ← rt.readBinToEndInto rstrBegin
12          h.putStrLn s!"/-- generated from {path} -/"
13          h.putStrLn "def runtime : String :="
14          h.write $ runtime.pushMany rstrEnd

```

FFI or *foreign function interface*, refers to calling functions external to the language. Since *Tigris* compiles to SBCL, a rather high and low-level⁸⁶ language, FFI is obtained for *free* if one is familiar with the calling convention described before as we do not have to deal with memory management or boxing/unboxing. Metaprogramming is used extensively to make the use of FFI as convenient as possible. Since *Tigris* is not able to see an externally defined function, it puts every trust it has in the user whom is responsible for

making sure the function is correctly typed, both in LISP and in the extern declaration.

However, schemes declared this way can not be checked for their *sanity*, i.e., it is possible to inhabit the types that should otherwise be empty, or types that can only be proved (inhabited) with circular argument such as $\forall \alpha. \alpha$ or $\forall \alpha, \beta. \alpha \rightarrow \beta$. Though, the former already exists because of the deprecated fixpoint combinator `rec`, as in `rec idα`,⁸⁷ whereas the latter could be useful, though unsafely, when specialized to $\forall \alpha. \text{Unit} \rightarrow \alpha$ (as seen in an example later).

We illustrate the use of FFI through a somewhat elaborate example:

```

extern println "%println" : forall a, a -> Unit
extern append "%string-append" : String -> String -> String
extern toString "%to-string" : forall a, a -> String

```

⁸⁶Common Lisp is high-level in having GC and being a dynamically typed functional language; and is low-level in having basic special operators and a bunch of low-level features such as `go`, `block`, `progn` that dates back to 1950s.

⁸⁷while it damages the soundness of the type system, we keep it nonetheless for educational purposes. Besides, users can avoid it, since normal programming do not use `rec` at all.


```

extern read "%read" : forall a, Unit -> a
infixl 65 "++" => append

let fact n =
  let rec go acc
    | 0 => acc
    | m =>
      let _ = println $ "fact "
        ++ toString (n - m) ++ " = "
        ++ toString (acc * m)
      in go an @@ m - 1
  in go 1 n
postfix 50 "!" => fact

let main =
  let _ = println "Enter a number:" in
  let x = read () in
  x ! -- x! is also a valid identifier. So we parenthesize it/use juxtaposition.

```

read is an unsafe function whose return type can be specialized to any type, in this case, Int. Internally, a name associated to a FFI declaration is transformed, during parsing, to its FFI name, e.g., read becomes %read. Where the implementations of these external functions are

```

(defforeign |%to-string| (x _) ;; e.g. toString : ∀a, a -> String
  (declare (ignore gamma _))
  (funcall k (princ-to-string x)))
(defforeign "%string-append" (x y) ;; e.g. string-append : String -> String -> String
  (declare (ignore gamma) (type string x y))
  (funcall k (concatenate 'string x y)))
(defforeign |%read| (_ _) ;; e.g. read (unsafe) : ∀a, Unit -> a
  (declare (ignore gamma _ _))
  (funcall k (read)))
(defforeign |%println| (x _) ;; e.g. println : ∀a, a -> Unit
  (declare (ignore gamma _))
  (princ x) (terpri)
  (funcall k nil))

```

Compiling and running the program results in⁸⁸

```

1 $ tigrisl examples/fact.tig --link ffi.lisp -o dist-fact.fasl && chmod +x dist-fact.fasl
2 ; wrote (...)/dist-fact.fasl
3 ; compilation finished in 0:00:00.021
4 $ ./dist-fact.fasl
5 Enter a number:
6 > 5
7 fact 0 = 5
8 fact 1 = 20
9 fact 2 = 60
10 fact 3 = 120

```

⁸⁸though ffi.lisp in this case is automatically linked. Linking it again with --link would make SBCL complain about duplicate definitions. --link here is only to demonstrate its use.

```

11 fact 4 = 120
12 120

```

For users with LISP experience, they will quickly notice the unfamiliar notation `defforeign`. This, together with other gadgets, are LISP macros to make the definition of foreign functions easier, which lives inside `ffi.lisp`. Amongst them, the most important are

- `WITH-ARGS` destructures `payload = (cons α Γ)` and binds each user argument to the symbols provided and a special variable `GAMMA` denoting the captured environment. Basically, it generates the prelude part of a function where `payload` are destructured according to the provided *lambda list*⁸⁹. This macro is not supposed to be used along; It is intended to be called by `defforeign` and users should use that instead. `with-args` is defined as

```

1  (defmacro with-args (payload arg-list &body body)
2    (let* ((alpha (gensym "ALPHA"))
3           (tmp   (gensym "TMP")))
4      (labels ((build-binds (args)
5                  (cond
6                    ((null args) '())
7                    ((null (cdr args)) `((, (car args) (car ,tmp))))
8                    (t
9                     (let ((head (car args))
10                          (rest (cdr args)))
11                       (append `((,head (car ,tmp))
12                                (,tmp (cdr ,tmp)))
13                               (if (null (cdr rest))
14                                   `((,(car rest) ,tmp))
15                                   (build-binds rest)))))))
16      `(let* ((,alpha (car ,payload))
17              (gamma (cdr ,payload))
18              (,tmp ,alpha)
19              ,@(build-binds arg-list))
20        (declare (ignorable gamma))
21        ,@body))))

```

- `DEFFOREIGN` defines a tigris-callable foreign function. It is modeled after `defun` and combines `defparameter`/`with-args`. A foreign function definition takes the form of⁹⁰

defforeign *function-name ffi-lambda-list {form}* → closure-object*

Aside from `GAMMA`, it additionally binds `k` i.e. the continuation, applying `k` can be thought as returning the value of the function as this is the canonical way to do so. `function-name`, or `name` in the implementation below, is the name of the foreign function, can be a string `"%println"` (preserves case), or a symbol `|%println|`. Users are highly recommended to escape the symbol (to preserve cases) as LISP by default does not distinguish cases but *Tigris* does. `defforeign` is implemented as

⁸⁹LISP jargon. Refers to a list that specifies a set of parameters (sometimes called lambda variables) and a protocol for receiving values for those parameters.

⁹⁰Here we use the same BNF syntax from X3J13/CLtL2. Unexperienced users may interpret it as: `(defforeign function-name (x y) forms)` that returns a closure object, where `(x y)` is `ffi-lambda-list` representing function parameters that may vary in length; `forms` refers to a list of forms that is the function body.

```

1  (defmacro defforeign (name arg-list &body body)
2    (flet ((name-to-string (x)
3              (etypecase x
4                (string x)
5                (symbol (symbol-name x)))))
6      (escape-bar (s)
7        (with-output-to-string (out)
8          (loop for c across s do
9            (case c
10              (#\| (write-string "\\|" out))
11              (#\\ (write-string "\\\\" out))
12              (t (write-char c out))))))
13      (make-closure (f)
14        `(cons C (vector
15                  (function ,f)
16                  (cons E (vector))))))
17      (let* ((ns (name-to-string name))
18             (fn-sym (intern (escape-bar ns)))
19             (cell (intern ns))
20             (clos (make-closure fn-sym)))
21        `(eval-when (:compile-toplevel :load-toplevel :execute)
22          (defun ,fn-sym (payload k)
23            (with-args payload ,arg-list ,@body))
24            (defparameter ,cell ,clos))))))

```

Note that the use of special operator `eval-when` is to make sure that any foreign functions defined in this way gets compiled and bound when `(load ..)`.

With these macros, defining foreign functions should be as easy as defining normal LISP functions.

caution! Any function output by the SBCL backend has the following compiler instructions:

```
(declare (optimize (speed 3) (safety 0) (debug 0)))
```

which removes most safety features of LISP, maximizing performance. Any error can result in a compromised LISP image (memory environment) with unreadable low-level error messages. It is best to remove this instruction when debugging.

tigrisl is the CLI interface to interact with the compiler. **tigrisl** uses a simple hand-written recursive descent parser to parse command line arguments; Multiple input files are compiled in parallel with the same amount of threads launched. An invocation takes the form of

```
tigrisl [flags] ifiles [-o ofiles]
```

where *ifiles* and *ofiles* refers to a list of input and output files that should either

- matches one to one. As in, for all i , input file name f_i^{in} , there always exists a output file name f_i^{out} , vice versa.
- matches one to one, with residual *ifiles*. As in, for *ifiles* $f_1^{\text{in}}, \dots, f_i^{\text{in}}, f_{i+1}^{\text{in}}, \dots, f_n^{\text{in}}$ and *ofiles* $f_1^{\text{out}}, \dots, f_i^{\text{out}}$, we have $f_1^{\text{in}}, \dots, f_i^{\text{in}}$ matching $f_1^{\text{out}}, \dots, f_i^{\text{out}}$, with residual *ifiles* $f_{i+1}^{\text{in}}, f_n^{\text{in}}$ using default output naming convention i.e. appending with `.lisp`.

- any other situation results in a exception.

...and *notable flags* are

- `--lam`, emit raw *Lambda* IR with optimizations;
- `--cc`, emit CC'd *Lambda* IR;
- `--cps`, emit CPS IR;
- `--sysf`, emit System F IR; These four flags can be used together.
- `-nl`, `--no-lisp`, do not emit Common Lisp source.
- `-ne`, `--no-entry`, do not generate entrypoint, where entrypoint is also called driver code, that is the following:

```
(defun |__start| ()
  (format nil "~A"
    (funcall #'|main| nil #'identity)))
```

- `--legacy`, use deprecated typechecker/IR/backend. Not recommended;
- `-l`, `--link <path>`, link additional lisp source from `<path>`, can be used multiple times. Default behavior: link `ffi.lean`. Note that, internally, linking a file `f` adds a `(load f)` to the top of the object code. The order of multiple `--links` is preserved here.
- `-o <ofiles>`, *positional vararg*, anything beyond this point is considered a part of `-o`. `<ofiles>` has file extension-directed output where `*.fasl` generate FASL executables and anything else for LISP sources. FASL generation is done by concatenating every linked LISP files together with the generated object code, then feed them into the launched SBCL with *block compilation* on to generate a monolithic binary. This binary can usually run directly in shell. If that is not the case, users may also use `sbcl --script <fasl>` to run fasl files.
- `--fasl`, emit FASL, applies to all input files.

5 PP: The *dependently-typed* table printer

What's the fun when there is *Tigrisd*, the theorem prover counterpart of *Tigris*, but the implementation of them doesn't involve some dependent type practices – Actually, both CLI interfaces and REPLs use a dependently-typed table printer that lives inside the PP module, not to mention the termination proofs of some recursive functions.

The author, and this project advocates for the use of dependent types in programming and below, we'll discuss the design and implementation of this simple printer, hopefully, to provide some insights into how dependent types can help users catch more unexpected errors at compile-time, rather than runtime.

We wish the types of table are distinguished by their headers, say, the header (name, gender, age) should be different from (product name, manufacturer, serial number, price). To achieve this at compile-time, we shall rely on the typechecker. Suppose the header is a list, we may turn it into a *dynamically-sized*⁹¹ product *type* whose length and component type must match for the product types to be unified (Text.SString is stylized string):

```
abbrev ToProd (as : List α) (motive : Type -> Type) : Type :=
  match as with
  | [] => PUnit
```

⁹¹In the sense that its size depends on the length of the header.

```

| [_] ⇒ motive α
| _ :: x :: xs ⇒
  motive α × ToProd (x :: xs) motive

```

```
variable (header : List Text.SString)
```

```
abbrev Row := ToProd header id
```

The above reads: $\text{ToProd } (as : \text{List } \alpha) f$ can be eliminated to one of 1) PUnit^{92} , 2) $f \alpha$, or 3) $f \alpha \times \text{ToProd } (\text{cdr } as) f$, based on the *shape* of as :

1. where as is empty;
2. where as is a *singleton*;
3. where as contains *two or more* elements.

This effectively tells us that ToProd can *eliminate* to one of the three⁹³ types depending on the value of as , specifically, its shape. Readers should pay great attention to this approach, or thought process, as the pattern matching used here is interpreted differently⁹⁴ (more of a *recursor*-based interpretation),⁹⁵ which is mirrored later.

Note that *motive* is type theory jargon. It represents the target type (or a part of it) we want it to be eliminated to. Informally, it can be best compared to the function used when mapping some data structures. Also, the use of **abbrev** rather than **def** here guarantees the definition to be *reducible*, not *semireducible* or *irreducible*, allowing the automatic unfolding of the definition when using tactics such as `rfl`, short for reflexivity.

Now, we may encode an entry (a `Row`) of a table as ToProd header id . This way, we require the entry of some table to match the shape of its header, otherwise it is a type error; And the table type itself, obviously, can be defined as

```
abbrev TableOf := Array $ Row header
```

We use `Array` here because it is overridden in the runtime with *real continuous data structures* rather than a linked list `List`. Besides, it also makes sense as we want *the shape* of the entry to match its header, but there should not be any size restriction when it comes to the size of a table, i.e. the numbers of entry a table has.

Furthermore, since we are making a *printer*, we want the output to look good, or at least, looks like a table. Here we define

```

inductive Alignment | left | center | right
inductive PadsBy | perCell | perCol
abbrev Align := ToProd header λ _ ⇒ Alignment
abbrev OverrideWidth := ToProd header λ _ ⇒ Option Nat

```

where

- `Align` is a tuple of the same shape as a header, denoting each column's alignment;
- `PadsBy` is the padding strategy. `perCell` makes every cell to have the same size as the largest one in them. This is rarely useful, but may look good if the table isn't too large; `perCol` is the one that is actually useful and more familiar to users since it applies the same strategy but local to each column.

⁹² *universe-polymorphic* version of `Unit`. Though since the result lives in $\text{Type} \equiv \text{Type } 0 \equiv \text{Sort } 1$, it is effectively `Unit`.

⁹³ To be precise, it is $2 + n$ for as of length n .

⁹⁴ different than a *operational* interpretation, in the sense of regular programming.

⁹⁵ And it is indeed in the form of recursor invocations inside the Lean *kernel*.

```

<ofiles>      a list of output files written to
file extension-directed output:
- '*.fasl' for FASL output, regardless of --fasl
- anything else for LISP output

```

```

(.str "<ofiles>", .str "a list of output files written to"
(.str "", .by1 "file extension-directed output:")
(.str "", .str " - '*.fasl' for FASL output, regardless of"
(.str "", .str " - anything else for LISP output")

```

Figure 5: output of the table printer and its equivalent table entry

- `OverrideWidth`, despite its name, stores the calculated width of each column. It is named `OverrideWidth` because it can be overridden (*before* the calculation) to have more manual control over the output, providing better granularity.

Based on this, we sketch the configuration of this printer as

```

structure PPSpec (header : List Text.SString) where
  align      : Align header
  width      : OverrideWidth header := 0
  header?    : Bool := true
  margin     : Nat := 3
  padsBy     : PadsBy := .perCol
  truncate   : Bool := false

```

where `margin` is *sum* of a cell’s left and right margin. And `truncate`, which is a flag that toggles the skip of a cell if it is empty i.e. gets replaced by the cell to its right (also respects margin); Despite its simple nature, this is actually more interesting than one might imagine, as this could be used to create indentation, together with `margin`, to achieve some interesting effects, such as simulating tool tips, which is most useful when printing command helppages as shown in Figure 5.

Now, we can actually implement this printer by first defining some helper functions that calculates the width of every column: Figure 6. This is where the aforementioned thought process come into use. The type `ToProd`, can somewhat be interpreted as having multiple values (though this “value” are *types*), as it *depends* on its `List` argument, though, it remains as one type at a moment because it is statically typed. We illustrate the usage of this thought process by introducing a concept called *discriminant refinement*.

Discriminant refinement is, a jargon, though fairly straightforward, stating that the type information of a match discriminant can be *refined* based on the pattern matching against it. An example being `match x with | 0 => ...`⁹⁶, where Lean knows that within this branch, `x` must be `0`, or as a *hypothesis* that becomes part of the *ambient types*: $h : x = 0$. – A very reasonable, and true take. This technique, is used fairly a lot in programming with dependent types, as is in this case: we *guide* Lean the underlying type of `ToProd` through patterns against `header`:

1. a `[]`, stating `header = []`, refines `ToProd` to `PUnit`, as stated in its definition;
2. a `[_]`, stating `header = [_]`, a singleton, or, a `List` with one element of type α , refines `ToProd` to $f\alpha$, as stated in its definition;
3. an Erlang-like `%[_ , _ | _]`, which is syntactic sugar for `_ :: _ :: _`, stating `header = . :: (. :: .)`, a list with two or more elements, refines `ToProd` to a dynamically-sized product.

Where f is the motive, in this case, the constant function that maps all types to `Option Nat`. The point of these seemingly useless patterns differs from the regular purpose of providing templates

⁹⁶Though, in non-dependent type context, this feature may need to be enabled manually, in the form of `match h : x with ...`, by prefixing `x` with a name `h`, for binding the hypotheses in each branch in order to for them be picked up by the typechecker.

```

def ovToStr (ov : OverrideWidth header)
  : String := match header with
  | [] => "" | [_] => toString ov
  | %[_ , _ | _] => toString ov.1 ++ ovToStr ov.2

def zeroOverride (header : List Text.SString)
  : OverrideWidth header := match header with
  | [] => () | [_] => none
  | %[_ , y | ys] => (none, zeroOverride (y :: ys))

def maxN (h1 h2 : OverrideWidth header)
  : OverrideWidth header := match header with
  | [] => () | [_] => max h1 h2
  | %[_ , _ | _] => (max h1.1 h2.1, maxN h1.2 h2.2)

def merge (h1 h2 : OverrideWidth header)
  : OverrideWidth header := match header with
  | [] => () | [_] => max h1 h2
  | %[_ , _ | _] => (h2.1 <| h1.1, merge h1.2 h2.2)

instance : ToString $ OverrideWidth header := <ovToStr>
instance : Union $ OverrideWidth header := <merge>
instance : Max $ OverrideWidth header := <maxN>
instance : Zero $ OverrideWidth header := <zeroOverride header>

```

Figure 6: Helper functions used to calculate column width

for destructuring; Instead, it is to provide additional information to the typechecker, which, if not omitted, results in a type error as Lean cannot deduce the type of header on its own. It is the user's responsibility to do case-by-case analysis just as in proofs. From the outsider's perspective, it is as simple as to mirror the cases in the type definition.

Remark (Common usage of discriminant refinement). Furthermore, discriminant refinement deals with surrounding environment. `foldr1` on `List` usually does not permit empty lists, applying to such is a runtime error in Haskell, for example. In Lean, we simply make this requirement an assumption and defines the function as:

```

def foldr1 (f : α -> α -> α) (as : List α) (h : as ≠ []) : α :=
  match as with
  | [] => h rfl >.elim -- or simply remove this branch.
  | [x] => x
  | x :: y :: xs => f x (foldr1 f (y :: xs) nofun)

```

where h is refined to $[] \neq []$ in the empty branch where we manually *proved false* and used *ex falso* to match the expected type. Though, this branch can simply be removed as Lean can automatically proves it `False`. Similarly `nofun` is a shorthand used here to prove $y :: xs \neq []$, but the proof itself automatically handled by Lean. Users calling this function must prove that as is nonempty through actual proofs or simply introducing a hypothesis using `if h : as ≠ [] then ...`. Though, we shall stop here as this is beyond the scope of this text, which is not a tutorial.

Getting back to implementing table printer, we found ourself having several polymorphic helpers in the sense that they are independent of header and we may use it to implement some operations like `toString`, $(\cdot \cup \cdot)$, `max` and `0` where it denotes the zero element of some algebraic structure. From there, implementing printers becomes trivial.

This concludes the description of *Tigris*.

6 *Tigrisd*: a lightweight theorem prover

Tigrisd is a experimental dependently-typed theorem prover that demonstrates clear, modular and exemplary theorem prover design. *Tigrisd*, together with *Tigris*, is a bundled solution serving educational purposes that hopes to become useful in programming language/theorem proving education.

6.1 The basics

The syntax of *Tigrisd* is very similar to *Tigris*, with a few exceptions, especially in relation to its dependently-typed nature. The implementation of this syntax also uses monadic parsing, in the form of parser combinators, specifically, the *layout* combinators, which use off-side rules (i.e. indentation aware). This decision is made this way because proofs are generally more verbose than programs. Some of the connectives can be omit if we use indentation-aware parsing, which, is more closer to a human-readable form. We list some notable syntax changes below.

Indexed family has the form

```

<dtype-decl> ::= inductive <dtype-name> <dtype-binder>* : <dtype-univ>
               "where"? ("|" <dtype-branch>)*
<dtype-branch> ::= <dtype-ctor> : sepBy1("-", <dtype-binder>)
<dtype-ctor>   ::= <upper-ident>
<dtype-name>   ::= <upper-ident>
<dtype-binder> ::= <dtype-tbinder> | <dtype-ibinder>
<dtype-tbinder> ::= (<ident> : <term>)
<dtype-ibinder> ::= {<ident> : <term>}
<dtype-univ>   ::= Type

```

An concrete index family example in *Tigrisd* is

```

inductive Vector (a : Type) (n : Nat) : Type where
| Nil : Vector a Zero
| Cons : {m : Nat} -> a -> Vector a (Succ m)

```

With type irrelevant binders wrapped in curly brackets {} i.e. not used in programming. A constructor's result type is checked against the type that is being defined to see if their *head* matches, in the case, they must be an application of Vector.

This example defines a List that encodes its length in the type, similar to the Vector in Lean.

function now takes the form of:

```

<tm-lam> ::= fun <dtype-binder>+ => <tm>
<tm-let> ::= let <ident> <dtype-binder>* : <tm> => <tm>

```

The let-binding syntactic sugar described in *Tigris* applies here:

```

let map {α : Type} {β : Type} {n : Nat} (f : α -> β) (as : Vector α n) : Vector β n =
  match as with
| Nil      => Nil
| Cons m x xs => Cons m (f x) (map α β m f xs)

```

Note that currently *Tigrisd* does not perform bidirectional type inference or implicit inference, which requires users to provide every relevant binders. Readers should pay great attention to this new property:

function takes types just the same as taking values, since types are terms as well.

Which, yields some interesting rules that previously, as in *Tigris*, requires extensions, such as skolem variables. Consider replacing the recursive call of `map` with `xs`, i.e. `Cons m (f x) xs`, which, immediately yields a type error because α and β are distinctive parameters. The typechecker cannot establish that `Vector  $\alpha$  n` equals `Vector  $\beta$  n`, unless the equality $h : \alpha = \beta$ ⁹⁷ is proven, only after which said types can be proved equal through α -equivalence; In this case, it is not and it can't be, as α and β are arbitrary types and there is not any assumption that entails h .

7 Summary

⁹⁷Equality itself is a big topic. In this case, we are talking about definitional equality. Detailed information on this is described later.